

Guía de reciclaje de componentes web - Funcionalidades 10 a 25

Extensión del documento 1. Mantiene el mismo formato: explicación, HTML, CSS, JavaScript modular, JavaScript normal, variables que debes cambiar, pasos, errores comunes y checklist.

Este documento sigue el orden de la tabla de contenido del recurso unificado. En esa tabla, la funcionalidad 10 es Contador animado y la 25 es Acordeón. Por eso esta guía cubre exactamente ese bloque para que puedas continuar después del documento 1 sin perder la numeración original.

#	Funcionalidad	Uso principal
10	Contador animado	Cuenta visualmente desde cero hasta un valor objetivo. Es útil para secciones de métricas, dashboards, estadísticas, resultados, landing pages y tarje...
11	Login demo	Crea una pantalla o tarjeta de inicio de sesión para practicar formularios, validación, cambios de estado y almacenamiento de sesión. En un proyecto r...
12	Logout demo	Cierra la sesión simulada y limpia datos temporales del navegador. Se complementa con el login demo y con componentes que muestran contenido solo cuan...
13	Recordar usuario	Guarda el correo, nombre o preferencia del usuario para rellenar campos automáticamente. Es común en formularios de login, preferencias, temas visuale...
14	Password toggle	Permite mostrar u ocultar una contraseña. Es una funcionalidad muy común en formularios de login, registro y cambio de contraseña.
15	Validación de formulario	Valida campos obligatorios, correo, longitud mínima y reglas personalizadas antes de procesar o enviar datos.
16	Mensajes por campo	Muestra errores o ayudas debajo de cada input. Complementa la validación y hace que el usuario sepa exactamente qué corregir.
17	Character counter	Cuenta los caracteres de un textarea o input. Se usa mucho en comentarios, bios, descripciones, formularios de contacto y publicaciones.
18	Range preview	Muestra en tiempo real el valor de un input type="range". Es útil para filtros de precio, volumen, porcentaje, rating, tamaños, cantidades y configura...
19	File info	Muestra el nombre, tamaño y tipo de archivo seleccionado. Es útil en formularios con carga de CV, imágenes, documentos, facturas o evidencias.
20	Autosave draft	Guarda automáticamente un borrador mientras el usuario escribe. Evita pérdida de información si se cierra la página por accidente.
21	Stepper / Wizard	Divide un proceso en pasos. Es común en formularios largos, registros, compras, onboarding y creación de perfiles.
22	Skeleton loader	Muestra una estructura de carga temporal antes de renderizar datos reales. Es muy usado en páginas modernas para que la espera se sienta más profesio...
23	Select personalizado	Reemplaza el select nativo por un selector bonito, controlable y editable. Permite opciones con estilo, flecha personalizada, blur y mejor integración...

#	Funcionalidad	Uso principal
24	Tabs / pestañas	Permite cambiar entre paneles internos sin recargar la página. Se usa en perfiles, productos, dashboards, documentación y configuraciones.
25	Acordeón / FAQ	Abre y cierra bloques de contenido. Es perfecto para preguntas frecuentes, secciones de ayuda, detalles de producto y menús informativos.

Cómo usar este documento

- Primero identifica la funcionalidad que quieres reutilizar según el número y el nombre.
- Copia el HTML mínimo en tu index.html o en la sección donde lo necesites.
- Copia el CSS en styles.css. Si ya tienes variables globales, adapta colores, radios y espacios.
- Elige solo una versión de JavaScript: modular con funciones.js + script.js, o normal con script-normal.js.
- Cambia los ids, clases y data-* exactamente donde se indique en la tabla de variables.
- Prueba la funcionalidad sola antes de mezclarla con otros componentes.

10. Contador animado

¿Qué hace?

Cuenta visualmente desde cero hasta un valor objetivo. Es útil para secciones de métricas, dashboards, estadísticas, resultados, landing pages y tarjetas tipo "clientes atendidos", "proyectos", "ventas" o "porcentaje de avance".

¿Cuándo usarlo?

Úsalo cuando quieras mostrar un número de forma más atractiva que un texto estático. También sirve para practicar manipulación del DOM, intervalos de tiempo y atributos data-.*.

Estructura que debes copiar

HTML mínimo

```
<section class="stats-grid">
  <article class="stat-card reveal">
    <span class="stat-card__number" data-counter data-target="150"
      data-suffix="+">0</span>
    <p>Proyectos realizados</p>
  </article>

  <article class="stat-card reveal">
    <span class="stat-card__number" data-counter data-target="98"
      data-suffix="%">0</span>
    <p>Satisfacción</p>
  </article>
</section>
```

CSS necesario

```
.stats-grid {
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(180px, 1fr));
  gap: 1rem;
}

.stat-card {
  padding: 1.4rem;
  border-radius: 1.2rem;
  background: rgba(255, 255, 255, 0.12);
  border: 1px solid rgba(255, 255, 255, 0.22);
  backdrop-filter: blur(14px);
  text-align: center;
}

.stat-card__number {
  display: block;
  font-size: clamp(2rem, 5vw, 3rem);
  font-weight: 800;
  color: var(--primary, #2563eb);
}
```

funciones.js - versión modular con export

```
export class AnimatedCounters {
  constructor(selector = "[data-counter]", duration = 1200) {
    this.counters = document.querySelectorAll(selector);
    this.duration = duration;
    this.start();
  }

  start() {
    this.counters.forEach((counter) => this.animate(counter));
  }

  animate(counter) {
    const target = Number(counter.dataset.target);
    const suffix = counter.dataset.suffix || "";
    const startTime = performance.now();

    const update = (currentTime) => {
      const elapsed = currentTime - startTime;
      const progress = Math.min(elapsed / this.duration, 1);
      const value = Math.floor(progress * target);

      counter.textContent = `${value}${suffix}`;

      if (progress < 1) {
        requestAnimationFrame(update);
      }
    };

    requestAnimationFrame(update);
  }
}
```

script.js - importación e inicialización

```
import { AnimatedCounters } from "./funciones.js";

new AnimatedCounters("[data-counter]", 1400);
```

script-normal.js - versión normal sin módulos

```
const counters = document.querySelectorAll("[data-counter]");

counters.forEach((counter) => {
  const target = Number(counter.dataset.target);
  const suffix = counter.dataset.suffix || "";
  let current = 0;
  const step = Math.ceil(target / 60);

  const interval = setInterval(() => {
    current += step;
    if (current >= target) {
      current = target;
      clearInterval(interval);
    }
    counter.textContent = `${current}${suffix}`;
  }, 20);
});
```

Variables, clases e IDs que puedes cambiar

Elemento	Para qué sirve	Cómo adaptarlo
data-target	Número final del contador.	Cámbialo por el valor que quieras mostrar.
data-suffix	Símbolo que aparece al final.	Puedes usar +, %, K, M, COP o dejarlo vacío.
.stat-card	Tarjeta visual de la métrica.	Puedes cambiar colores, fondo, blur y bordes.
duration	Duración de la animación.	Aumenta el valor para que cuente más lento.

Paso a paso para reciclarlo en otro proyecto

1. Copia el HTML del contador dentro de la sección donde mostrarás las métricas.
2. Ajusta data-target según el número final que necesitas.
3. Agrega o quita data-suffix según quieras mostrar %, + u otro símbolo.
4. Copia el CSS de .stats-grid y .stat-card en tu archivo styles.css.
5. En funciones.js exporta la clase AnimatedCounters.
6. En script.js importa la clase y crea una instancia con new AnimatedCounters().
7. Prueba que los números avancen al cargar la página.

Qué debes revisar al integrarlo

- Que los ids del HTML coincidan con los ids usados en JavaScript.
- Que no estés usando el mismo id dos veces en la misma página.
- Que las clases activas como is-active, is-open, is-valid o is-invalid estén escritas igual en HTML, CSS y JS.
- Que si usas módulos el script se cargue con type="module".
- Que si usas la versión normal no dejes imports ni exports en el archivo.

Errores comunes

- Poner data-target con texto en vez de número.
- Olvidar importar la clase si usas módulos.
- Usar demasiados contadores con intervalos pesados; requestAnimationFrame es más fluido.

Cómo cambiar nombres sin romper la funcionalidad

Paso	Qué hacer	Recomendación
1	Cambia primero el id en el HTML.	Ejemplo: loginForm por formAcceso.
2	Busca ese mismo id en script.js o script-normal.js.	Debe quedar exactamente igual.
3	Si el CSS usa una clase, cambia la clase en HTML y CSS al mismo tiempo.	No cambies solo uno de los dos archivos.
4	Si hay data-* como data-tab, data-step o data-value, revisa qué lo lee en JS.	Los atributos data-* suelen ser el puente entre HTML y JS.
5	Prueba la funcionalidad aislada antes de mezclarla con otros componentes.	Así sabes si el error viene del componente o de otra parte.

Adaptaciones comunes en proyectos reales

- Para una landing page: usa Contador animado dentro de una sección con clase .section y manten el diseño centrado.
- Para un dashboard: coloca Contador animado dentro de una card reutilizable y controla el estado con clases como is-active o is-open.
- Para un formulario: valida ids, required, minlength y data-error antes de enviar datos.
- Para un proyecto con muchos componentes: inicializa esta funcionalidad desde una función init10() o desde initApp().
- Para trabajo en equipo: deja comentarios cortos indicando qué HTML, CSS y JS pertenecen a Contador animado.

Mini guía de depuración si no funciona

1. Abre la consola del navegador con F12 y revisa si aparece un error rojo.
2. Si el error dice null, significa que JavaScript no encontró un elemento del DOM; revisa ids y clases.
3. Si el diseño no cambia, revisa que la clase que agrega JavaScript exista también en CSS.

4. Si usas import/export, abre el proyecto con Live Server; no lo abras solo con doble clic.
5. Si el componente depende de localStorage o sessionStorage, limpia el almacenamiento del navegador y prueba de nuevo.
6. Comenta temporalmente otros componentes para confirmar que no hay conflicto de nombres.

Patrón mental para reutilizarlo

Cada funcionalidad de esta guía se puede entender como una combinación de tres capas: la capa HTML define la estructura, la capa CSS define el diseño visual y la capa JavaScript controla el comportamiento. Para reciclarla bien, no copies solo una capa: copia la estructura mínima, los estilos necesarios y la inicialización correspondiente. Después, cambia nombres y valores de forma ordenada.

Consejo de adaptación profesional

Si quieres que el contador se active solo cuando aparece en pantalla, combínalo con IntersectionObserver o con la funcionalidad Reveal on scroll del documento anterior.

11. Login demo

¿Qué hace?

Crea una pantalla o tarjeta de inicio de sesión para practicar formularios, validación, cambios de estado y almacenamiento de sesión. En un proyecto real se conecta con backend, API o Firebase.

¿Cuándo usarlo?

Úsalo en portafolios, dashboards, intranets de práctica, paneles administrativos y proyectos donde quieras simular ingreso de usuario.

Estructura que debes copiar

HTML mínimo

```
<section class="auth-card" id="loginSection">
  <h2>Iniciar sesión</h2>

  <form id="loginForm" novalidate>
    <label for="loginEmail">Correo</label>
    <input id="loginEmail" type="email" placeholder="correo@ejemplo.com" required>
    <small class="field-error" data-error="loginEmail"></small>

    <label for="loginPassword">Contraseña</label>
    <input id="loginPassword" type="password" placeholder="Mínimo 6 caracteres"
      required minlength="6">
    <small class="field-error" data-error="loginPassword"></small>

    <button type="submit">Entrar</button>
  </form>

  <p id="loginMessage" class="form-message"></p>
</section>
```

CSS necesario

```
.auth-card {
  max-width: 420px;
  margin: 2rem auto;
  padding: 2rem;
  border-radius: 1.5rem;
  background: rgba(255, 255, 255, 0.12);
  border: 1px solid rgba(255, 255, 255, 0.2);
  backdrop-filter: blur(16px);
}

.auth-card label {
  display: block;
  margin-top: 1rem;
  margin-bottom: .4rem;
  font-weight: 700;
}

.auth-card input {
  width: 100%;
  padding: .85rem 1rem;
  border-radius: .9rem;
  border: 1px solid rgba(255, 255, 255, 0.28);
  background: rgba(255, 255, 255, 0.12);
  color: inherit;
}

.field-error {
  display: block;
  min-height: 1rem;
  margin-top: .3rem;
  color: #fca5a5;
  font-size: .82rem;
}

.form-message.is-success { color: #86efac; }
.form-message.is-error { color: #fca5a5; }
```

funciones.js - versión modular con export

```
export class LoginDemo {
  constructor(formId, messageId) {
    this.form = document.getElementById(formId);
    this.message = document.getElementById(messageId);
    this.init();
  }

  init() {
    this.form.addEventListener("submit", (event) => {
      event.preventDefault();
      this.login();
    });
  }

  login() {
    const email = this.form.querySelector("#loginEmail").value.trim();
    const password = this.form.querySelector("#loginPassword").value.trim();

    if (!email || !password) {
      this.showMessage("Completa todos los campos.", "error");
      return;
    }

    if (password.length < 6) {
      this.showMessage("La contraseña debe tener mínimo 6 caracteres.",
        "error");
      return;
    }

    const user = { email, loggedAt: new Date().toISOString() };
    sessionStorage.setItem("kit-session-user", JSON.stringify(user));
    this.showMessage("Ingreso correcto. Sesión demo guardada.", "success");
  }

  showMessage(text, type) {
    this.message.textContent = text;
    this.message.className = `form-message is-${type}`;
  }
}
```

script.js - importación e inicialización

```
import { LoginDemo } from "./funciones.js";

new LoginDemo("loginForm", "loginMessage");
```

script-normal.js - versión normal sin módulos

```
const loginForm = document.getElementById("loginForm");
const loginMessage = document.getElementById("loginMessage");

loginForm.addEventListener("submit", (event) => {
  event.preventDefault();

  const email = document.getElementById("loginEmail").value.trim();
  const password = document.getElementById("loginPassword").value.trim();

  if (!email || !password) {
    loginMessage.textContent = "Completa todos los campos.";
    loginMessage.className = "form-message is-error";
    return;
  }

  sessionStorage.setItem("kit-session-user", JSON.stringify({ email }));
  loginMessage.textContent = "Ingreso correcto.";
  loginMessage.className = "form-message is-success";
});
```

Variables, clases e IDs que puedes cambiar

Elemento	Para qué sirve	Cómo adaptarlo
#loginForm	Formulario que escucha el submit.	Cambia el id si tu formulario tiene otro nombre.
#loginEmail	Campo de correo.	Debe coincidir con el selector usado en JS.
#loginPassword	Campo de contraseña.	Puedes cambiar minlength según tu regla.
kit-session-user	Clave de sessionStorage.	Cámbiala si usas varios proyectos.

Paso a paso para reciclarlo en otro proyecto

1. Copia la tarjeta de login en la página donde quieres simular el acceso.
2. Verifica que el formulario tenga id="loginForm" o ajusta el id en JavaScript.
3. Conecta el CSS de .auth-card, .field-error y .form-message.
4. Si usas módulos, exporta LoginDemo en funciones.js e impórtalo en script.js.
5. Prueba enviar el formulario vacío, con contraseña corta y con datos válidos.
6. Para producción reemplaza la simulación por una llamada fetch a tu backend.

Qué debes revisar al integrarlo

- Que los ids del HTML coincidan con los ids usados en JavaScript.
- Que no estés usando el mismo id dos veces en la misma página.
- Que las clases activas como is-active, is-open, is-valid o is-invalid estén escritas igual en HTML, CSS y JS.
- Que si usas módulos el script se cargue con type="module".
- Que si usas la versión normal no dejes imports ni exports en el archivo.

Errores comunes

- Creer que este login es seguridad real.
- No usar event.preventDefault() y que la página se recargue.
- Repetir ids si hay más de un login.

Cómo cambiar nombres sin romper la funcionalidad

Paso	Qué hacer	Recomendación
1	Cambia primero el id en el HTML.	Ejemplo: loginForm por formAcceso.
2	Busca ese mismo id en script.js o script-normal.js.	Debe quedar exactamente igual.
3	Si el CSS usa una clase, cambia la clase en HTML y CSS al mismo tiempo.	No cambies solo uno de los dos archivos.
4	Si hay data-* como data-tab, data-step o data-value, revisa qué lo lee en JS.	Los atributos data-* suelen ser el puente entre HTML y JS.
5	Prueba la funcionalidad aislada antes de mezclarla con otros componentes.	Así sabes si el error viene del componente o de otra parte.

Adaptaciones comunes en proyectos reales

- Para una landing page: usa Login demo dentro de una sección con clase .section y manten el diseño centrado.
- Para un dashboard: coloca Login demo dentro de una card reutilizable y controla el estado con clases como is-active o is-open.

- Para un formulario: valida ids, required, minlength y data-error antes de enviar datos.
- Para un proyecto con muchos componentes: inicializa esta funcionalidad desde una función `init11()` o desde `initApp()`.
- Para trabajo en equipo: deja comentarios cortos indicando qué HTML, CSS y JS pertenecen a Login demo.

Mini guía de depuración si no funciona

1. Abre la consola del navegador con F12 y revisa si aparece un error rojo.
2. Si el error dice null, significa que JavaScript no encontró un elemento del DOM; revisa ids y clases.
3. Si el diseño no cambia, revisa que la clase que agrega JavaScript exista también en CSS.
4. Si usas import/export, abre el proyecto con Live Server; no lo abras solo con doble clic.
5. Si el componente depende de localStorage o sessionStorage, limpia el almacenamiento del navegador y prueba de nuevo.
6. Comenta temporalmente otros componentes para confirmar que no hay conflicto de nombres.

Patrón mental para reutilizarlo

Cada funcionalidad de esta guía se puede entender como una combinación de tres capas: la capa HTML define la estructura, la capa CSS define el diseño visual y la capa JavaScript controla el comportamiento. Para reciclarla bien, no copies solo una capa: copia la estructura mínima, los estilos necesarios y la inicialización correspondiente. Después, cambia nombres y valores de forma ordenada.

Consejo de adaptación profesional

No guardes contraseñas en localStorage ni sessionStorage. Para producción usa backend, tokens seguros, HTTPS y validación del servidor.

12. Logout demo

¿Qué hace?

Cierra la sesión simulada y limpia datos temporales del navegador. Se complementa con el login demo y con componentes que muestran contenido solo cuando existe usuario.

¿Cuándo usarlo?

Úsalo en dashboards de práctica, paneles demo y proyectos donde quieras simular la entrada y salida de un usuario.

Estructura que debes copiar

HTML mínimo

```
<div class="user-panel">
  <span id="userStatus">Sin sesión activa</span>
  <button id="logoutButton" type="button">Cerrar sesión</button>
</div>
```

CSS necesario

```
.user-panel {
  display: flex;
  align-items: center;
  justify-content: space-between;
  gap: 1rem;
  padding: 1rem;
  border-radius: 1rem;
  background: rgba(255, 255, 255, 0.1);
}

#logoutButton {
  border: 0;
  border-radius: .8rem;
  padding: .75rem 1rem;
  cursor: pointer;
  color: #fff;
  background: linear-gradient(135deg, #ef4444, #f97316);
}
```

funciones.js - versión modular con export

```
export class LogoutDemo {
  constructor(buttonId, statusId, storageKey = "kit-session-user") {
    this.button = document.getElementById(buttonId);
    this.status = document.getElementById(statusId);
    this.storageKey = storageKey;
    this.init();
  }

  init() {
    this.renderStatus();
    this.button.addEventListener("click", () => this.logout());
  }

  renderStatus() {
    const user = JSON.parse(sessionStorage.getItem(this.storageKey));
    this.status.textContent = user ? `Sesión: ${user.email}` : "Sin sesión activa";
  }

  logout() {
    sessionStorage.removeItem(this.storageKey);
    this.renderStatus();
  }
}
```

script.js - importación e inicialización

```
import { LogoutDemo } from "../funciones.js";

new LogoutDemo("logoutButton", "userStatus", "kit-session-user");
```

script-normal.js - versión normal sin módulos

```
const logoutButton = document.getElementById("logoutButton");
const userStatus = document.getElementById("userStatus");

function renderUserStatus() {
  const user = JSON.parse(sessionStorage.getItem("kit-session-user"));
  userStatus.textContent = user ? `Sesión: ${user.email}` : "Sin sesión activa";
}

logoutButton.addEventListener("click", () => {
  sessionStorage.removeItem("kit-session-user");
  renderUserStatus();
});

renderUserStatus();
```

Variables, clases e IDs que puedes cambiar

Elemento	Para qué sirve	Cómo adaptarlo
storageKey	Clave usada para guardar la sesión.	Debe ser la misma del login.
#logoutButton	Botón que cierra sesión.	Cambia el id si usas otro botón.
#userStatus	Texto donde se muestra el estado.	Puedes ponerlo en navbar o panel lateral.

Paso a paso para reciclarlo en otro proyecto

1. Agrega el panel o solo el botón de cerrar sesión donde lo necesites.
2. Asegúrate de usar la misma clave de sessionStorage que el login.
3. Copia el CSS o adapta el botón a tu diseño.
4. Inicializa la clase LogoutDemo después de LoginDemo.
5. Inicia sesión, recarga y verifica que el usuario aparezca.
6. Presiona cerrar sesión y revisa que el estado se actualice.

Qué debes revisar al integrarlo

- Que los ids del HTML coincidan con los ids usados en JavaScript.
- Que no estés usando el mismo id dos veces en la misma página.
- Que las clases activas como is-active, is-open, is-valid o is-invalid estén escritas igual en HTML, CSS y JS.
- Que si usas módulos el script se cargue con type="module".
- Que si usas la versión normal no dejes imports ni exports en el archivo.

Errores comunes

- Usar localStorage y sessionStorage mezclados sin control.
- Eliminar una clave diferente a la que guarda el login.
- No actualizar visualmente el estado después de cerrar sesión.

Cómo cambiar nombres sin romper la funcionalidad

Paso	Qué hacer	Recomendación
1	Cambia primero el id en el HTML.	Ejemplo: loginForm por formAcceso.

Paso	Qué hacer	Recomendación
2	Busca ese mismo id en script.js o script-normal.js.	Debe quedar exactamente igual.
3	Si el CSS usa una clase, cambia la clase en HTML y CSS al mismo tiempo.	No cambies solo uno de los dos archivos.
4	Si hay data-* como data-tab, data-step o data-value, revisa qué lo lee en JS.	Los atributos data-* suelen ser el puente entre HTML y JS.
5	Prueba la funcionalidad aislada antes de mezclarla con otros componentes.	Así sabes si el error viene del componente o de otra parte.

Adaptaciones comunes en proyectos reales

- Para una landing page: usa Logout demo dentro de una sección con clase .section y manten el diseño centrado.
- Para un dashboard: coloca Logout demo dentro de una card reutilizable y controla el estado con clases como is-active o is-open.
- Para un formulario: valida ids, required, minlength y data-error antes de enviar datos.
- Para un proyecto con muchos componentes: inicializa esta funcionalidad desde una función init12() o desde initApp().
- Para trabajo en equipo: deja comentarios cortos indicando qué HTML, CSS y JS pertenecen a Logout demo.

Mini guía de depuración si no funciona

1. Abre la consola del navegador con F12 y revisa si aparece un error rojo.
2. Si el error dice null, significa que JavaScript no encontró un elemento del DOM; revisa ids y clases.
3. Si el diseño no cambia, revisa que la clase que agrega JavaScript exista también en CSS.
4. Si usas import/export, abre el proyecto con Live Server; no lo abras solo con doble clic.
5. Si el componente depende de localStorage o sessionStorage, limpia el almacenamiento del navegador y prueba de nuevo.
6. Comenta temporalmente otros componentes para confirmar que no hay conflicto de nombres.

Patrón mental para reutilizarlo

Cada funcionalidad de esta guía se puede entender como una combinación de tres capas: la capa HTML define la estructura, la capa CSS define el diseño visual y la capa JavaScript controla el comportamiento. Para reciclarla bien, no copies solo una capa: copia la estructura mínima, los estilos necesarios y la inicialización correspondiente. Después, cambia nombres y valores de forma ordenada.

Consejo de adaptación profesional

Después de cerrar sesión en producción también debes invalidar el token del servidor o borrar cookies seguras si las usas.

13. Recordar usuario

¿Qué hace?

Guarda el correo, nombre o preferencia del usuario para rellenar campos automáticamente. Es común en formularios de login, preferencias, temas visuales y configuraciones.

¿Cuándo usarlo?

Úsalo cuando quieras mejorar experiencia de usuario sin pedir los mismos datos cada vez.

Estructura que debes copiar

HTML mínimo

```
<label class="remember-row">
  <input type="checkbox" id="rememberUser">
  Recordar mi correo
</label>
```

CSS necesario

```
.remember-row {
  display: flex;
  align-items: center;
  gap: .6rem;
  margin-top: .8rem;
  font-size: .9rem;
  color: var(--text-muted, #cbd5e1);
}

.remember-row input {
  width: 18px;
  height: 18px;
  accent-color: var(--primary, #2563eb);
}
```

funciones.js - versión modular con export

```
export class RememberUser {
  constructor(emailInputId, checkboxId, storageKey = "kit-remember-email") {
    this.emailInput = document.getElementById(emailInputId);
    this.checkbox = document.getElementById(checkboxId);
    this.storageKey = storageKey;
    this.init();
  }

  init() {
    const savedEmail = localStorage.getItem(this.storageKey);

    if (savedEmail) {
      this.emailInput.value = savedEmail;
      this.checkbox.checked = true;
    }
  }

  saveIfChecked() {
    if (this.checkbox.checked) {
      localStorage.setItem(this.storageKey, this.emailInput.value.trim());
    } else {
      localStorage.removeItem(this.storageKey);
    }
  }
}
```

script.js - importación e inicialización

```
import { RememberUser } from "../funciones.js";

const rememberUser = new RememberUser("loginEmail", "rememberUser");

// Dentro del submit del login, después de validar:
rememberUser.saveIfChecked();
```

script-normal.js - versión normal sin módulos

```
const rememberCheckbox = document.getElementById("rememberUser");
const emailInput = document.getElementById("loginEmail");
const savedEmail = localStorage.getItem("kit-remember-email");

if (savedEmail) {
  emailInput.value = savedEmail;
  rememberCheckbox.checked = true;
}

function saveRememberedEmail() {
  if (rememberCheckbox.checked) {
    localStorage.setItem("kit-remember-email", emailInput.value.trim());
  } else {
    localStorage.removeItem("kit-remember-email");
  }
}
```

Variables, clases e IDs que puedes cambiar

Elemento	Para qué sirve	Cómo adaptarlo
#rememberUser	Checkbox de recordar usuario.	Cambia el id si tienes varios formularios.
#loginEmail	Input que se rellena automáticamente.	Debe ser el campo de correo real.
kit-remember-email	Clave de localStorage.	Personalízala por proyecto.

Paso a paso para reciclarlo en otro proyecto

1. Agrega el checkbox debajo del correo en el login.
2. Copia el CSS para que el checkbox se alinee bien.
3. Crea la instancia de RememberUser en script.js.
4. Dentro del submit, llama saveIfChecked() después de validar los datos.
5. Prueba marcar el checkbox, iniciar sesión y recargar la página.
6. Prueba desmarcarlo y verificar que el correo ya no se guarde.

Qué debes revisar al integrarlo

- Que los ids del HTML coincidan con los ids usados en JavaScript.
- Que no estés usando el mismo id dos veces en la misma página.
- Que las clases activas como is-active, is-open, is-valid o is-invalid estén escritas igual en HTML, CSS y JS.
- Que si usas módulos el script se cargue con type="module".
- Que si usas la versión normal no dejes imports ni exports en el archivo.

Errores comunes

- Guardar contraseñas en localStorage: no se debe hacer.
- Llamar saveIfChecked antes de validar el correo.
- No cambiar la clave si mezclas varios proyectos.

Cómo cambiar nombres sin romper la funcionalidad

Paso	Qué hacer	Recomendación
1	Cambia primero el id en el HTML.	Ejemplo: loginForm por formAcceso.
2	Busca ese mismo id en script.js o script-normal.js.	Debe quedar exactamente igual.
3	Si el CSS usa una clase, cambia la clase en HTML y CSS al mismo tiempo.	No cambies solo uno de los dos archivos.
4	Si hay data-* como data-tab, data-step o data-value, revisa qué lo lee en JS.	Los atributos data-* suelen ser el puente entre HTML y JS.
5	Prueba la funcionalidad aislada antes de mezclarla con otros componentes.	Así sabes si el error viene del componente o de otra parte.

Adaptaciones comunes en proyectos reales

- Para una landing page: usa Recordar usuario dentro de una sección con clase .section y manten el diseño centrado.
- Para un dashboard: coloca Recordar usuario dentro de una card reutilizable y controla el estado con clases como is-active o is-open.
- Para un formulario: valida ids, required, minlength y data-error antes de enviar datos.
- Para un proyecto con muchos componentes: inicializa esta funcionalidad desde una función init13() o desde initApp().
- Para trabajo en equipo: deja comentarios cortos indicando qué HTML, CSS y JS pertenecen a Recordar usuario.

Mini guía de depuración si no funciona

1. Abre la consola del navegador con F12 y revisa si aparece un error rojo.
2. Si el error dice null, significa que JavaScript no encontró un elemento del DOM; revisa ids y clases.
3. Si el diseño no cambia, revisa que la clase que agrega JavaScript exista también en CSS.
4. Si usas import/export, abre el proyecto con Live Server; no lo abras solo con doble clic.
5. Si el componente depende de localStorage o sessionStorage, limpia el almacenamiento del navegador y prueba de nuevo.
6. Comenta temporalmente otros componentes para confirmar que no hay conflicto de nombres.

Patrón mental para reutilizarlo

Cada funcionalidad de esta guía se puede entender como una combinación de tres capas: la capa HTML define la estructura, la capa CSS define el diseño visual y la capa JavaScript controla el comportamiento. Para reciclarla bien, no copies solo una capa: copia la estructura mínima, los estilos necesarios y la inicialización correspondiente. Después, cambia nombres y valores de forma ordenada.

Consejo de adaptación profesional

Solo recuerda datos no sensibles. Recordar un correo es aceptable; recordar contraseñas en el navegador con JS no lo es.

14. Password toggle

¿Qué hace?

Permite mostrar u ocultar una contraseña. Es una funcionalidad muy común en formularios de login, registro y cambio de contraseña.

¿Cuándo usarlo?

Úsalo cuando tengas inputs type="password" y quieras mejorar la experiencia del usuario.

Estructura que debes copiar

HTML mínimo

```
<div class="password-field">
  <input id="loginPassword" type="password" placeholder="Contraseña">
  <button type="button" data-toggle-password="loginPassword">Mostrar</button>
</div>
```

CSS necesario

```
.password-field {
  display: flex;
  align-items: center;
  gap: .5rem;
}

.password-field input {
  flex: 1;
}

.password-field button {
  width: auto;
  min-height: auto;
  padding: .75rem .9rem;
  border-radius: .8rem;
  border: 1px solid rgba(255, 255, 255, 0.2);
  background: rgba(255, 255, 255, 0.12);
  color: inherit;
}
```

funciones.js - versión modular con export

```
export class PasswordToggle {
  constructor(selector = "[data-toggle-password]") {
    this.buttons = document.querySelectorAll(selector);
    this.init();
  }

  init() {
    this.buttons.forEach((button) => {
      button.addEventListener("click", () => this.toggle(button));
    });
  }

  toggle(button) {
    const inputId = button.dataset.togglePassword;
    const input = document.getElementById(inputId);
    const isHidden = input.type === "password";

    input.type = isHidden ? "text" : "password";
    button.textContent = isHidden ? "Ocultar" : "Mostrar";
  }
}
```

script.js - importación e inicialización

```
import { PasswordToggle } from "./funciones.js";

new PasswordToggle("[data-toggle-password]");
```

script-normal.js - versión normal sin módulos

```
document.querySelectorAll("[data-toggle-password]").forEach((button) => {
  button.addEventListener("click", () => {
    const input = document.getElementById(button.dataset.togglePassword);
    const isHidden = input.type === "password";

    input.type = isHidden ? "text" : "password";
    button.textContent = isHidden ? "Ocultar" : "Mostrar";
  });
});
```

Variables, clases e IDs que puedes cambiar

Elemento	Para qué sirve	Cómo adaptarlo
data-toggle-password	Debe contener el id del input password.	Si el input cambia de id, actualiza este valor.
#loginPassword	Input que cambia entre password y text.	Puedes usarlo en cualquier formulario.
.password-field	Contenedor visual.	Puedes modificar flex, gap y colores.

Paso a paso para reciclarlo en otro proyecto

1. Encierra el input y el botón dentro de .password-field.
2. El botón debe tener data-toggle-password con el id del input.
3. Copia el CSS para alinear el botón con el input.
4. Exporta PasswordToggle o copia la versión normal.
5. Prueba que el botón cambie de Mostrar a Ocultar.
6. Verifica que el valor de la contraseña no se borre al alternar.

Qué debes revisar al integrarlo

- Que los ids del HTML coincidan con los ids usados en JavaScript.
- Que no estés usando el mismo id dos veces en la misma página.
- Que las clases activas como is-active, is-open, is-valid o is-invalid estén escritas igual en HTML, CSS y JS.
- Que si usas módulos el script se cargue con type="module".
- Que si usas la versión normal no dejes imports ni exports en el archivo.

Errores comunes

- Poner data-toggle-password sin valor.
- Usar un id que no existe.
- Crear varios inputs con el mismo id.

Cómo cambiar nombres sin romper la funcionalidad

Paso	Qué hacer	Recomendación
1	Cambia primero el id en el HTML.	Ejemplo: loginForm por formAcceso.
2	Busca ese mismo id en script.js o script-normal.js.	Debe quedar exactamente igual.
3	Si el CSS usa una clase, cambia la clase en HTML y CSS al mismo tiempo.	No cambies solo uno de los dos archivos.
4	Si hay data-* como data-tab, data-step o data-value, revisa qué lo lee en JS.	Los atributos data-* suelen ser el puente entre HTML y JS.

Paso	Qué hacer	Recomendación
5	Prueba la funcionalidad aislada antes de mezclarla con otros componentes.	Así sabes si el error viene del componente o de otra parte.

Adaptaciones comunes en proyectos reales

- Para una landing page: usa Password toggle dentro de una sección con clase `.section` y manten el diseño centrado.
- Para un dashboard: coloca Password toggle dentro de una card reutilizable y controla el estado con clases como `is-active` o `is-open`.
- Para un formulario: valida `ids`, `required`, `minlength` y `data-error` antes de enviar datos.
- Para un proyecto con muchos componentes: inicializa esta funcionalidad desde una función `init14()` o desde `initApp()`.
- Para trabajo en equipo: deja comentarios cortos indicando qué HTML, CSS y JS pertenecen a Password toggle.

Mini guía de depuración si no funciona

1. Abre la consola del navegador con F12 y revisa si aparece un error rojo.
2. Si el error dice `null`, significa que JavaScript no encontró un elemento del DOM; revisa `ids` y clases.
3. Si el diseño no cambia, revisa que la clase que agrega JavaScript exista también en CSS.
4. Si usas `import/export`, abre el proyecto con Live Server; no lo abras solo con doble clic.
5. Si el componente depende de `localStorage` o `sessionStorage`, limpia el almacenamiento del navegador y prueba de nuevo.
6. Comenta temporalmente otros componentes para confirmar que no hay conflicto de nombres.

Patrón mental para reutilizarlo

Cada funcionalidad de esta guía se puede entender como una combinación de tres capas: la capa HTML define la estructura, la capa CSS define el diseño visual y la capa JavaScript controla el comportamiento. Para reciclarla bien, no copies solo una capa: copia la estructura mínima, los estilos necesarios y la inicialización correspondiente. Después, cambia nombres y valores de forma ordenada.

Consejo de adaptación profesional

En móviles, este patrón mejora mucho la experiencia porque reduce errores al escribir contraseñas.

15. Validación de formulario

¿Qué hace?

Valida campos obligatorios, correo, longitud mínima y reglas personalizadas antes de procesar o enviar datos.

¿Cuándo usarlo?

Úsalo en login, registro, contacto, cotizaciones, compras, formularios de soporte y paneles administrativos.

Estructura que debes copiar

HTML mínimo

```
<form id="contactForm" novalidate>
  <label for="contactName">Nombre</label>
  <input id="contactName" type="text" required minlength="3">
  <small class="field-error" data-error="contactName"></small>

  <label for="contactEmail">Correo</label>
  <input id="contactEmail" type="email" required>
  <small class="field-error" data-error="contactEmail"></small>

  <button type="submit">Enviar</button>
</form>
```

CSS necesario

```
input.is-invalid,
textarea.is-invalid {
  border-color: #ef4444;
  box-shadow: 0 0 4px rgba(239, 68, 68, .16);
}

input.is-valid,
textarea.is-valid {
  border-color: #22c55e;
  box-shadow: 0 0 4px rgba(34, 197, 94, .14);
}

.field-error {
  color: #fca5a5;
  font-size: .82rem;
  min-height: 1rem;
}
```

funciones.js - versión modular con export

```

export class FormValidator {
  constructor(formId) {
    this.form = document.getElementById(formId);
    this.fields = this.form.querySelectorAll("input, textarea, select");
    this.init();
  }

  init() {
    this.form.addEventListener("submit", (event) => {
      event.preventDefault();

      if (this.validateAll()) {
        console.log("Formulario válido");
      }
    });

    this.fields.forEach((field) => {
      field.addEventListener("input", () => this.validateField(field));
    });
  }

  validateAll() {
    return [...this.fields].every((field) => this.validateField(field));
  }

  validateField(field) {
    let message = "";

    if (field.hasAttribute("required") && field.value.trim() === "") {
      message = "Este campo es obligatorio.";
    } else if (field.type === "email" && !field.value.includes("@")) {
      message = "Ingresa un correo válido.";
    } else if (field.minLength > 0 && field.value.length < field.minLength) {
      message = `Mínimo ${field.minLength} caracteres.`;
    }

    this.showError(field, message);
    return message === "";
  }

  showError(field, message) {
    const error = this.form.querySelector(`[data-error="${field.id}"]`);
    field.classList.toggle("is-invalid", Boolean(message));
    field.classList.toggle("is-valid", !message && field.value.trim() !== "");
    if (error) error.textContent = message;
  }
}

```

script.js - importación e inicialización

```

import { FormValidator } from "./funciones.js";

new FormValidator("contactForm");

```

script-normal.js - versión normal sin módulos

```
const contactForm = document.getElementById("contactForm");
const fields = contactForm.querySelectorAll("input, textarea, select");

function validateField(field) {
  let message = "";

  if (field.required && field.value.trim() === "") message = "Este campo es obligatorio.";
  if (field.type === "email" && !field.value.includes("@")) message = "Correo inválido.";

  const error = contactForm.querySelector(`[data-error="${field.id}"]`);
  field.classList.toggle("is-invalid", Boolean(message));
  if (error) error.textContent = message;

  return message === "";
}

contactForm.addEventListener("submit", (event) => {
  event.preventDefault();
  const valid = [...fields].every(validateField);
  if (valid) console.log("Formulario válido");
});
```

Variables, clases e IDs que puedes cambiar

Elemento	Para qué sirve	Cómo adaptarlo
required	Indica que el campo no puede ir vacío.	Agrégallo solo a campos obligatorios.
minlength	Longitud mínima.	Cambia el número según tu necesidad.
data-error	Relaciona el mensaje con el id del input.	Debe coincidir exactamente con el id.
.is-invalid / .is-valid	Clases visuales.	Cambia colores y sombra en CSS.

Paso a paso para reciclarlo en otro proyecto

1. Agrega required, type="email" o minlength donde corresponda.
2. Debajo de cada input agrega un small con data-error igual al id del campo.
3. Copia los estilos de campos válidos e inválidos.
4. Exporta FormValidator en funciones.js.
5. Inicializa new FormValidator("idDelFormulario") en script.js.
6. Prueba enviar vacío, con email incorrecto y con datos correctos.

Qué debes revisar al integrarlo

- Que los ids del HTML coincidan con los ids usados en JavaScript.
- Que no estés usando el mismo id dos veces en la misma página.
- Que las clases activas como is-active, is-open, is-valid o is-invalid estén escritas igual en HTML, CSS y JS.
- Que si usas módulos el script se cargue con type="module".
- Que si usas la versión normal no dejes imports ni exports en el archivo.

Errores comunes

- Olvidar novalidate si quieres controlar mensajes con JS.
- Usar data-error diferente al id del campo.
- Validar solo al submit y no en input; la experiencia se siente peor.

Cómo cambiar nombres sin romper la funcionalidad

Paso	Qué hacer	Recomendación
1	Cambia primero el id en el HTML.	Ejemplo: loginForm por formAcceso.
2	Busca ese mismo id en script.js o script-normal.js.	Debe quedar exactamente igual.
3	Si el CSS usa una clase, cambia la clase en HTML y CSS al mismo tiempo.	No cambies solo uno de los dos archivos.
4	Si hay data-* como data-tab, data-step o data-value, revisa qué lo lee en JS.	Los atributos data-* suelen ser el puente entre HTML y JS.
5	Prueba la funcionalidad aislada antes de mezclarla con otros componentes.	Así sabes si el error viene del componente o de otra parte.

Adaptaciones comunes en proyectos reales

- Para una landing page: usa Validación de formulario dentro de una sección con clase .section y manten el diseño centrado.
- Para un dashboard: coloca Validación de formulario dentro de una card reutilizable y controla el estado con clases como is-active o is-open.
- Para un formulario: valida ids, required, minlength y data-error antes de enviar datos.
- Para un proyecto con muchos componentes: inicializa esta funcionalidad desde una función init15() o desde initApp().
- Para trabajo en equipo: deja comentarios cortos indicando qué HTML, CSS y JS pertenecen a Validación de formulario.

Mini guía de depuración si no funciona

1. Abre la consola del navegador con F12 y revisa si aparece un error rojo.
2. Si el error dice null, significa que JavaScript no encontró un elemento del DOM; revisa ids y clases.
3. Si el diseño no cambia, revisa que la clase que agrega JavaScript exista también en CSS.
4. Si usas import/export, abre el proyecto con Live Server; no lo abras solo con doble clic.
5. Si el componente depende de localStorage o sessionStorage, limpia el almacenamiento del navegador y prueba de nuevo.
6. Comenta temporalmente otros componentes para confirmar que no hay conflicto de nombres.

Patrón mental para reutilizarlo

Cada funcionalidad de esta guía se puede entender como una combinación de tres capas: la capa HTML define la estructura, la capa CSS define el diseño visual y la capa JavaScript controla el comportamiento. Para reciclarla bien, no copies solo una capa: copia la estructura mínima, los estilos necesarios y la inicialización correspondiente. Después, cambia nombres y valores de forma ordenada.

Consejo de adaptación profesional

La validación del frontend mejora la experiencia, pero no reemplaza la validación del backend.

16. Mensajes por campo

¿Qué hace?

Muestra errores o ayudas debajo de cada input. Complementa la validación y hace que el usuario sepa exactamente qué corregir.

¿Cuándo usarlo?

Úsalo en formularios profesionales donde cada campo debe tener retroalimentación individual.

Estructura que debes copiar

HTML mínimo

```
<div class="field">
  <label for="username">Usuario</label>
  <input id="username" type="text" required>
  <small class="field-help">Usa letras y números.</small>
  <small class="field-error" data-error="username"></small>
</div>
```

CSS necesario

```
.field {
  margin-bottom: 1rem;
}

.field-help {
  display: block;
  margin-top: .3rem;
  color: var(--text-muted, #94a3b8);
  font-size: .78rem;
}

.field-error {
  display: block;
  min-height: 1rem;
  margin-top: .25rem;
  color: #fb7185;
  font-size: .8rem;
}
```

funciones.js - versión modular con export

```
export function setFieldMessage(form, fieldId, message = "") {
  const errorBox = form.querySelector(`[data-error="${fieldId}"]`);
  const field = form.querySelector(`#${fieldId}`);

  if (errorBox) errorBox.textContent = message;
  if (field) field.classList.toggle("is-invalid", Boolean(message));
}

export function clearFieldMessages(form) {
  form.querySelectorAll("[data-error]").forEach((errorBox) => {
    errorBox.textContent = "";
  });

  form.querySelectorAll(".is-invalid").forEach((field) => {
    field.classList.remove("is-invalid");
  });
}
```

script.js - importación e inicialización

```
import { setFieldMessage, clearFieldMessages } from "./funciones.js";

const form = document.getElementById("contactForm");
setFieldMessage(form, "username", "El usuario es obligatorio.");
// clearFieldMessages(form);
```


script-normal.js - versión normal sin módulos

```
function setFieldMessage(form, fieldId, message = "") {
  const errorBox = form.querySelector(`[data-error="${fieldId}"]`);
  const field = form.querySelector(`#${fieldId}`);

  if (errorBox) errorBox.textContent = message;
  if (field) field.classList.toggle("is-invalid", Boolean(message));
}
```

Variables, clases e IDs que puedes cambiar

Elemento	Para qué sirve	Cómo adaptarlo
data-error	Conecta el mensaje con un input.	Debe tener el mismo valor que el id del input.
.field-help	Texto de ayuda normal.	Úsalo para explicar reglas antes de que haya error.
.field-error	Texto de error.	Solo se llena cuando algo está mal.

Paso a paso para reciclarlo en otro proyecto

1. Agrupa cada input dentro de un div.field.
2. Agrega field-help si quieres instrucciones permanentes.
3. Agrega field-error con data-error igual al id del input.
4. Usa setFieldMessage(form, id, mensaje) desde tu validación.
5. Usa clearFieldMessages(form) antes de validar todo si quieres limpiar errores previos.

Qué debes revisar al integrarlo

- Que los ids del HTML coincidan con los ids usados en JavaScript.
- Que no estés usando el mismo id dos veces en la misma página.
- Que las clases activas como is-active, is-open, is-valid o is-invalid estén escritas igual en HTML, CSS y JS.
- Que si usas módulos el script se cargue con type="module".
- Que si usas la versión normal no dejes imports ni exports en el archivo.

Errores comunes

- Mostrar todos los errores en un solo párrafo genérico.
- No dejar espacio mínimo y que el diseño salte cuando aparece error.
- Usar mensajes muy largos que rompen la tarjeta.

Cómo cambiar nombres sin romper la funcionalidad

Paso	Qué hacer	Recomendación
1	Cambia primero el id en el HTML.	Ejemplo: loginForm por formAcceso.
2	Busca ese mismo id en script.js o script-normal.js.	Debe quedar exactamente igual.
3	Si el CSS usa una clase, cambia la clase en HTML y CSS al mismo tiempo.	No cambies solo uno de los dos archivos.
4	Si hay data-* como data-tab, data-step o data-value, revisa qué lo lee en JS.	Los atributos data-* suelen ser el puente entre HTML y JS.
5	Prueba la funcionalidad aislada antes de mezclarla con otros componentes.	Así sabes si el error viene del componente o de otra parte.

Adaptaciones comunes en proyectos reales

- Para una landing page: usa Mensajes por campo dentro de una sección con clase `.section` y manten el diseño centrado.
- Para un dashboard: coloca Mensajes por campo dentro de una card reutilizable y controla el estado con clases como `is-active` o `is-open`.
- Para un formulario: valida `ids`, `required`, `minlength` y `data-error` antes de enviar datos.
- Para un proyecto con muchos componentes: inicializa esta funcionalidad desde una función `init16()` o desde `initApp()`.
- Para trabajo en equipo: deja comentarios cortos indicando qué HTML, CSS y JS pertenecen a Mensajes por campo.

Mini guía de depuración si no funciona

1. Abre la consola del navegador con F12 y revisa si aparece un error rojo.
2. Si el error dice `null`, significa que JavaScript no encontró un elemento del DOM; revisa `ids` y clases.
3. Si el diseño no cambia, revisa que la clase que agrega JavaScript exista también en CSS.
4. Si usas `import/export`, abre el proyecto con Live Server; no lo abras solo con doble clic.
5. Si el componente depende de `localStorage` o `sessionStorage`, limpia el almacenamiento del navegador y prueba de nuevo.
6. Comenta temporalmente otros componentes para confirmar que no hay conflicto de nombres.

Patrón mental para reutilizarlo

Cada funcionalidad de esta guía se puede entender como una combinación de tres capas: la capa HTML define la estructura, la capa CSS define el diseño visual y la capa JavaScript controla el comportamiento. Para reciclarla bien, no copies solo una capa: copia la estructura mínima, los estilos necesarios y la inicialización correspondiente. Después, cambia nombres y valores de forma ordenada.

Consejo de adaptación profesional

Los mensajes por campo deben ser cortos y específicos. Evita textos genéricos como "error".

17. Character counter

¿Qué hace?

Cuenta los caracteres de un textarea o input. Se usa mucho en comentarios, bios, descripciones, formularios de contacto y publicaciones.

¿Cuándo usarlo?

Úsalo cuando haya un límite de caracteres y el usuario necesite verlo en tiempo real.

Estructura que debes copiar

HTML mínimo

```
<label for="bio">Descripción</label>
<textarea id="bio" maxlength="160" placeholder="Escribe una descripción
  corta"></textarea>
<small id="bioCounter">0 / 160</small>
```

CSS necesario

```
textarea {
  width: 100%;
  min-height: 120px;
  resize: vertical;
  padding: 1rem;
  border-radius: 1rem;
}

#bioCounter {
  display: block;
  margin-top: .3rem;
  text-align: right;
  color: var(--text-muted, #94a3b8);
}

#bioCounter.is-limit {
  color: #f97316;
  font-weight: 700;
}
```

funciones.js - versión modular con export

```
export class CharacterCounter {
  constructor(inputId, counterId) {
    this.input = document.getElementById(inputId);
    this.counter = document.getElementById(counterId);
    this.max = Number(this.input.getAttribute("maxlength")) || 0;
    this.init();
  }

  init() {
    this.update();
    this.input.addEventListener("input", () => this.update());
  }

  update() {
    const length = this.input.value.length;
    this.counter.textContent = this.max ? `${length} / ${this.max}` : `${length}`;
    this.counter.classList.toggle("is-limit", this.max && length >= this.max *
      0.9);
  }
}
```

script.js - importación e inicialización

```
import { CharacterCounter } from "./funciones.js";

new CharacterCounter("bio", "bioCounter");
```

script-normal.js - versión normal sin módulos

```
const bio = document.getElementById("bio");
const bioCounter = document.getElementById("bioCounter");
const max = Number(bio.getAttribute("maxLength"));

function updateCounter() {
  const length = bio.value.length;
  bioCounter.textContent = `${length} / ${max}`;
  bioCounter.classList.toggle("is-limit", length >= max * 0.9);
}

bio.addEventListener("input", updateCounter);
updateCounter();
```

Variables, clases e IDs que puedes cambiar

Elemento	Para qué sirve	Cómo adaptarlo
maxlength	Límite real del textarea.	Cámbialo según el máximo permitido.
#bioCounter	Elemento donde se muestra el conteo.	Cambia el id si tienes varios contadores.
.is-limit	Estado cercano al límite.	Puedes cambiarlo a rojo o naranja.

Paso a paso para reciclarlo en otro proyecto

1. Agrega maxlength al input o textarea.
2. Crea un small donde se verá el contador.
3. Copia el CSS de contador y estado is-limit.
4. Inicializa CharacterCounter con el id del campo y el id del contador.
5. Escribe hasta llegar cerca del límite y revisa que cambie el color.

Qué debes revisar al integrarlo

- Que los ids del HTML coincidan con los ids usados en JavaScript.
- Que no estés usando el mismo id dos veces en la misma página.
- Que las clases activas como is-active, is-open, is-valid o is-invalid estén escritas igual en HTML, CSS y JS.
- Que si usas módulos el script se cargue con type="module".
- Que si usas la versión normal no dejes imports ni exports en el archivo.

Errores comunes

- Olvidar maxlength y no tener límite real.
- Usar innerHTML para un texto que no necesita HTML.
- No llamar update() al cargar y que aparezca vacío.

Cómo cambiar nombres sin romper la funcionalidad

Paso	Qué hacer	Recomendación
1	Cambia primero el id en el HTML.	Ejemplo: loginForm por formAcceso.
2	Busca ese mismo id en script.js o script-normal.js.	Debe quedar exactamente igual.
3	Si el CSS usa una clase, cambia la clase en HTML y CSS al mismo tiempo.	No cambies solo uno de los dos archivos.
4	Si hay data-* como data-tab, data-step o data-value, revisa qué lo lee en JS.	Los atributos data-* suelen ser el puente entre HTML y JS.

Paso	Qué hacer	Recomendación
5	Prueba la funcionalidad aislada antes de mezclarla con otros componentes.	Así sabes si el error viene del componente o de otra parte.

Adaptaciones comunes en proyectos reales

- Para una landing page: usa Character counter dentro de una sección con clase `.section` y manten el diseño centrado.
- Para un dashboard: coloca Character counter dentro de una card reutilizable y controla el estado con clases como `is-active` o `is-open`.
- Para un formulario: valida `ids`, `required`, `minlength` y `data-error` antes de enviar datos.
- Para un proyecto con muchos componentes: inicializa esta funcionalidad desde una función `init17()` o desde `initApp()`.
- Para trabajo en equipo: deja comentarios cortos indicando qué HTML, CSS y JS pertenecen a Character counter.

Mini guía de depuración si no funciona

1. Abre la consola del navegador con F12 y revisa si aparece un error rojo.
2. Si el error dice `null`, significa que JavaScript no encontró un elemento del DOM; revisa `ids` y clases.
3. Si el diseño no cambia, revisa que la clase que agrega JavaScript exista también en CSS.
4. Si usas `import/export`, abre el proyecto con Live Server; no lo abras solo con doble clic.
5. Si el componente depende de `localStorage` o `sessionStorage`, limpia el almacenamiento del navegador y prueba de nuevo.
6. Comenta temporalmente otros componentes para confirmar que no hay conflicto de nombres.

Patrón mental para reutilizarlo

Cada funcionalidad de esta guía se puede entender como una combinación de tres capas: la capa HTML define la estructura, la capa CSS define el diseño visual y la capa JavaScript controla el comportamiento. Para reciclarla bien, no copies solo una capa: copia la estructura mínima, los estilos necesarios y la inicialización correspondiente. Después, cambia nombres y valores de forma ordenada.

Consejo de adaptación profesional

Combina el contador con `maxlength` para que el límite sea real, no solo visual.

18. Range preview

¿Qué hace?

Muestra en tiempo real el valor de un input type="range". Es útil para filtros de precio, volumen, porcentaje, rating, tamaños, cantidades y configuraciones.

¿Cuándo usarlo?

Úsalo cuando un slider controle un valor que el usuario necesita leer claramente.

Estructura que debes copiar

HTML mínimo

```
<label for="priceRange">Precio máximo</label>
<input id="priceRange" type="range" min="0" max="100000" step="5000" value="50000">
<p>Valor seleccionado: <strong id="pricePreview">50000</strong></p>
```

CSS necesario

```
input[type="range"] {
  width: 100%;
  accent-color: var(--primary, #2563eb);
}

#pricePreview {
  color: var(--primary, #2563eb);
  font-size: 1.1rem;
}
```

funciones.js - versión modular con export

```
export class RangePreview {
  constructor(rangeId, previewId, formatter = null) {
    this.range = document.getElementById(rangeId);
    this.preview = document.getElementById(previewId);
    this.formatter = formatter || ((value) => value);
    this.init();
  }

  init() {
    this.update();
    this.range.addEventListener("input", () => this.update());
  }

  update() {
    this.preview.textContent = this.formatter(this.range.value);
  }
}
```

script.js - importación e inicialización

```
import { RangePreview } from "./funciones.js";

new RangePreview("priceRange", "pricePreview", (value) =>
  `$$${Number(value).toLocaleString("es-CO")}`);
```

script-normal.js - versión normal sin módulos

```
const priceRange = document.getElementById("priceRange");
const pricePreview = document.getElementById("pricePreview");

function updatePricePreview() {
  pricePreview.textContent = `$$${Number(priceRange.value).toLocaleString("es-CO")}`;
}

priceRange.addEventListener("input", updatePricePreview);
updatePricePreview();
```

Variables, clases e IDs que puedes cambiar

Elemento	Para qué sirve	Cómo adaptarlo
min / max / step	Rango y salto del slider.	Ajusta según precio, porcentaje o cantidad.
value	Valor inicial.	Debe estar entre min y max.
formatter	Función para formatear el resultado.	Úsala para COP, %, kg, L, etc.

Paso a paso para reciclarlo en otro proyecto

1. Copia el input type="range" y el elemento preview.
2. Ajusta min, max, step y value.
3. Inicializa RangePreview con los ids correctos.
4. Agrega formatter si quieres moneda o unidad.
5. Mueve el slider y revisa que el valor cambie sin recargar.

Qué debes revisar al integrarlo

- Que los ids del HTML coincidan con los ids usados en JavaScript.
- Que no estés usando el mismo id dos veces en la misma página.
- Que las clases activas como is-active, is-open, is-valid o is-invalid estén escritas igual en HTML, CSS y JS.
- Que si usas módulos el script se cargue con type="module".
- Que si usas la versión normal no dejes imports ni exports en el archivo.

Errores comunes

- No actualizar el preview al cargar.
- Dejar step demasiado grande o demasiado pequeño.
- Mostrar número sin formato cuando es precio.

Cómo cambiar nombres sin romper la funcionalidad

Paso	Qué hacer	Recomendación
1	Cambia primero el id en el HTML.	Ejemplo: loginForm por formAcceso.
2	Busca ese mismo id en script.js o script-normal.js.	Debe quedar exactamente igual.
3	Si el CSS usa una clase, cambia la clase en HTML y CSS al mismo tiempo.	No cambies solo uno de los dos archivos.
4	Si hay data-* como data-tab, data-step o data-value, revisa qué lo lee en JS.	Los atributos data-* suelen ser el puente entre HTML y JS.
5	Prueba la funcionalidad aislada antes de mezclarla con otros componentes.	Así sabes si el error viene del componente o de otra parte.

Adaptaciones comunes en proyectos reales

- Para una landing page: usa Range preview dentro de una sección con clase .section y manten el diseño centrado.
- Para un dashboard: coloca Range preview dentro de una card reutilizable y controla el estado con clases como is-active o is-open.
- Para un formulario: valida ids, required, minlength y data-error antes de enviar datos.
- Para un proyecto con muchos componentes: inicializa esta funcionalidad desde una función init18() o desde initApp().

- Para trabajo en equipo: deja comentarios cortos indicando qué HTML, CSS y JS pertenecen a Range preview.

Mini guía de depuración si no funciona

1. Abre la consola del navegador con F12 y revisa si aparece un error rojo.
2. Si el error dice null, significa que JavaScript no encontró un elemento del DOM; revisa ids y clases.
3. Si el diseño no cambia, revisa que la clase que agrega JavaScript exista también en CSS.
4. Si usas import/export, abre el proyecto con Live Server; no lo abras solo con doble clic.
5. Si el componente depende de localStorage o sessionStorage, limpia el almacenamiento del navegador y prueba de nuevo.
6. Comenta temporalmente otros componentes para confirmar que no hay conflicto de nombres.

Patrón mental para reutilizarlo

Cada funcionalidad de esta guía se puede entender como una combinación de tres capas: la capa HTML define la estructura, la capa CSS define el diseño visual y la capa JavaScript controla el comportamiento. Para reciclarla bien, no copies solo una capa: copia la estructura mínima, los estilos necesarios y la inicialización correspondiente. Después, cambia nombres y valores de forma ordenada.

Consejo de adaptación profesional

Siempre formatea el valor según contexto: moneda, porcentaje, litros, kilogramos o estrellas.

19. File info

¿Qué hace?

Muestra el nombre, tamaño y tipo de archivo seleccionado. Es útil en formularios con carga de CV, imágenes, documentos, facturas o evidencias.

¿Cuándo usarlo?

Úsalo siempre que tengas `input type="file"` y quieras que el usuario confirme qué archivo eligió.

Estructura que debes copiar

HTML mínimo

```
<label for="fileInput">Subir archivo</label>
<input id="fileInput" type="file" accept=".pdf,.png,.jpg,.jpeg">
<p id="fileInfo" class="file-info">Ningún archivo seleccionado.</p>
```

CSS necesario

```
.file-info {
  margin-top: .6rem;
  padding: .75rem 1rem;
  border-radius: .9rem;
  background: rgba(255, 255, 255, 0.1);
  color: var(--text-muted, #cbd5e1);
}
```

funciones.js - versión modular con export

```
export class FileInfo {
  constructor(inputId, infoId) {
    this.input = document.getElementById(inputId);
    this.info = document.getElementById(infoId);
    this.init();
  }

  init() {
    this.input.addEventListener("change", () => this.update());
  }

  update() {
    const file = this.input.files[0];

    if (!file) {
      this.info.textContent = "Ningún archivo seleccionado.";
      return;
    }

    const sizeKb = (file.size / 1024).toFixed(1);
    this.info.textContent = `${file.name} - ${sizeKb} KB - ${file.type || "tipo desconocido"}`;
  }
}
```

script.js - importación e inicialización

```
import { FileInfo } from "./funciones.js";

new FileInfo("fileInput", "fileInfo");
```

script-normal.js - versión normal sin módulos

```
const fileInput = document.getElementById("fileInput");
const fileInfo = document.getElementById("fileInfo");

fileInput.addEventListener("change", () => {
  const file = fileInput.files[0];

  if (!file) {
    fileInfo.textContent = "Ningún archivo seleccionado.";
    return;
  }

  fileInfo.textContent = `${file.name} - ${((file.size / 1024).toFixed(1))} KB`;
});
```

Variables, clases e IDs que puedes cambiar

Elemento	Para qué sirve	Cómo adaptarlo
accept	Tipos permitidos.	Cambia extensiones según tu caso.
file.size	Peso en bytes.	Puedes convertir a KB o MB.
file.type	Tipo MIME.	Sirve para validar imágenes o PDF.

Paso a paso para reciclarlo en otro proyecto

1. Agrega input type="file" y un contenedor de texto para la información.
2. Configura accept para limitar tipos de archivo.
3. Copia la clase FileInfo o la versión normal.
4. Prueba elegir archivo y cancelar selección.
5. Agrega validaciones de tamaño si el proyecto lo requiere.

Qué debes revisar al integrarlo

- Que los ids del HTML coincidan con los ids usados en JavaScript.
- Que no estés usando el mismo id dos veces en la misma página.
- Que las clases activas como is-active, is-open, is-valid o is-invalid estén escritas igual en HTML, CSS y JS.
- Que si usas módulos el script se cargue con type="module".
- Que si usas la versión normal no dejes imports ni exports en el archivo.

Errores comunes

- Creer que accept es seguridad total; también valida en backend.
- No manejar el caso cuando el usuario cancela.
- Mostrar bytes sin convertir y que sea difícil de leer.

Cómo cambiar nombres sin romper la funcionalidad

Paso	Qué hacer	Recomendación
1	Cambia primero el id en el HTML.	Ejemplo: loginForm por formAcceso.
2	Busca ese mismo id en script.js o script-normal.js.	Debe quedar exactamente igual.
3	Si el CSS usa una clase, cambia la clase en HTML y CSS al mismo tiempo.	No cambies solo uno de los dos archivos.
4	Si hay data-* como data-tab, data-step o data-value, revisa qué lo lee en JS.	Los atributos data-* suelen ser el puente entre HTML y JS.

Paso	Qué hacer	Recomendación
5	Prueba la funcionalidad aislada antes de mezclarla con otros componentes.	Así sabes si el error viene del componente o de otra parte.

Adaptaciones comunes en proyectos reales

- Para una landing page: usa File info dentro de una sección con clase .section y manten el diseño centrado.
- Para un dashboard: coloca File info dentro de una card reutilizable y controla el estado con clases como is-active o is-open.
- Para un formulario: valida ids, required, minlength y data-error antes de enviar datos.
- Para un proyecto con muchos componentes: inicializa esta funcionalidad desde una función init19() o desde initApp().
- Para trabajo en equipo: deja comentarios cortos indicando qué HTML, CSS y JS pertenecen a File info.

Mini guía de depuración si no funciona

1. Abre la consola del navegador con F12 y revisa si aparece un error rojo.
2. Si el error dice null, significa que JavaScript no encontró un elemento del DOM; revisa ids y clases.
3. Si el diseño no cambia, revisa que la clase que agrega JavaScript exista también en CSS.
4. Si usas import/export, abre el proyecto con Live Server; no lo abras solo con doble clic.
5. Si el componente depende de localStorage o sessionStorage, limpia el almacenamiento del navegador y prueba de nuevo.
6. Comenta temporalmente otros componentes para confirmar que no hay conflicto de nombres.

Patrón mental para reutilizarlo

Cada funcionalidad de esta guía se puede entender como una combinación de tres capas: la capa HTML define la estructura, la capa CSS define el diseño visual y la capa JavaScript controla el comportamiento. Para reciclarla bien, no copies solo una capa: copia la estructura mínima, los estilos necesarios y la inicialización correspondiente. Después, cambia nombres y valores de forma ordenada.

Consejo de adaptación profesional

Para cargas reales, valida también en backend el tipo y tamaño del archivo.

20. Autosave draft

¿Qué hace?

Guarda automáticamente un borrador mientras el usuario escribe. Evita pérdida de información si se cierra la página por accidente.

¿Cuándo usarlo?

Úsalo en comentarios largos, publicaciones, formularios, notas, mensajes y editores de contenido.

Estructura que debes copiar

HTML mínimo

```
<label for="draftText">Borrador</label>
<textarea id="draftText" placeholder="Escribe aquí..."></textarea>
<small id="draftStatus">Sin guardar todavía</small>
```

CSS necesario

```
#draftStatus {
  display: block;
  margin-top: .4rem;
  color: var(--text-muted, #94a3b8);
  font-size: .82rem;
}

#draftStatus.is-saved {
  color: #86efac;
}
```

funciones.js - versión modular con export

```
export class AutosaveDraft {
  constructor(inputId, statusId, storageKey = "kit-draft") {
    this.input = document.getElementById(inputId);
    this.status = document.getElementById(statusId);
    this.storageKey = storageKey;
    this.timeout = null;
    this.init();
  }

  init() {
    this.input.value = localStorage.getItem(this.storageKey) || "";
    this.input.addEventListener("input", () => this.scheduleSave());
  }

  scheduleSave() {
    this.status.textContent = "Guardando...";
    this.status.classList.remove("is-saved");
    clearTimeout(this.timeout);

    this.timeout = setTimeout(() => {
      localStorage.setItem(this.storageKey, this.input.value);
      this.status.textContent = "Borrador guardado automáticamente.";
      this.status.classList.add("is-saved");
    }, 600);
  }
}
```

script.js - importación e inicialización

```
import { AutosaveDraft } from "./funciones.js";

new AutosaveDraft("draftText", "draftStatus", "contact-draft");
```

script-normal.js - versión normal sin módulos

```
const draftText = document.getElementById("draftText");
const draftStatus = document.getElementById("draftStatus");
let draftTimeout;

draftText.value = localStorage.getItem("contact-draft") || "";

draftText.addEventListener("input", () => {
  draftStatus.textContent = "Guardando...";
  clearTimeout(draftTimeout);

  draftTimeout = setTimeout(() => {
    localStorage.setItem("contact-draft", draftText.value);
    draftStatus.textContent = "Borrador guardado.";
  }, 600);
});
```

Variables, clases e IDs que puedes cambiar

Elemento	Para qué sirve	Cómo adaptarlo
storageKey	Clave donde se guarda el borrador.	Cámbiala por formulario o página.
600	Tiempo de espera antes de guardar.	Aumenta si quieres guardar menos seguido.
#draftText	Textarea que se guarda.	Puedes usar input o textarea.

Paso a paso para reciclarlo en otro proyecto

1. Agrega textarea y un status visual.
2. Define una clave única de localStorage para ese borrador.
3. Copia AutosaveDraft e inicialízalo.
4. Escribe texto, espera y recarga la página.
5. Confirma que el texto se restaura automáticamente.
6. Agrega un botón de limpiar borrador si el flujo lo necesita.

Qué debes revisar al integrarlo

- Que los ids del HTML coincidan con los ids usados en JavaScript.
- Que no estés usando el mismo id dos veces en la misma página.
- Que las clases activas como is-active, is-open, is-valid o is-invalid estén escritas igual en HTML, CSS y JS.
- Que si usas módulos el script se cargue con type="module".
- Que si usas la versión normal no dejes imports ni exports en el archivo.

Errores comunes

- Usar la misma clave para varios formularios.
- Guardar información sensible sin avisar.
- Guardar en cada tecla sin debounce o setTimeout.

Cómo cambiar nombres sin romper la funcionalidad

Paso	Qué hacer	Recomendación
1	Cambia primero el id en el HTML.	Ejemplo: loginForm por formAcceso.
2	Busca ese mismo id en script.js o script-normal.js.	Debe quedar exactamente igual.
3	Si el CSS usa una clase, cambia la clase en HTML y CSS al mismo tiempo.	No cambies solo uno de los dos archivos.

Paso	Qué hacer	Recomendación
4	Si hay data-* como data-tab, data-step o data-value, revisa qué lo lee en JS.	Los atributos data-* suelen ser el puente entre HTML y JS.
5	Prueba la funcionalidad aislada antes de mezclarla con otros componentes.	Así sabes si el error viene del componente o de otra parte.

Adaptaciones comunes en proyectos reales

- Para una landing page: usa Autosave draft dentro de una sección con clase .section y manten el diseño centrado.
- Para un dashboard: coloca Autosave draft dentro de una card reutilizable y controla el estado con clases como is-active o is-open.
- Para un formulario: valida ids, required, minlength y data-error antes de enviar datos.
- Para un proyecto con muchos componentes: inicializa esta funcionalidad desde una función init20() o desde initApp().
- Para trabajo en equipo: deja comentarios cortos indicando qué HTML, CSS y JS pertenecen a Autosave draft.

Mini guía de depuración si no funciona

1. Abre la consola del navegador con F12 y revisa si aparece un error rojo.
2. Si el error dice null, significa que JavaScript no encontró un elemento del DOM; revisa ids y clases.
3. Si el diseño no cambia, revisa que la clase que agrega JavaScript exista también en CSS.
4. Si usas import/export, abre el proyecto con Live Server; no lo abras solo con doble clic.
5. Si el componente depende de localStorage o sessionStorage, limpia el almacenamiento del navegador y prueba de nuevo.
6. Comenta temporalmente otros componentes para confirmar que no hay conflicto de nombres.

Patrón mental para reutilizarlo

Cada funcionalidad de esta guía se puede entender como una combinación de tres capas: la capa HTML define la estructura, la capa CSS define el diseño visual y la capa JavaScript controla el comportamiento. Para reciclarla bien, no copies solo una capa: copia la estructura mínima, los estilos necesarios y la inicialización correspondiente. Después, cambia nombres y valores de forma ordenada.

Consejo de adaptación profesional

Agrega una opción de "descartar borrador" para que el usuario pueda limpiar el contenido guardado.

21. Stepper / Wizard

¿Qué hace?

Divide un proceso en pasos. Es común en formularios largos, registros, compras, onboarding y creación de perfiles.

¿Cuándo usarlo?

Úsalo cuando una tarea sea demasiado larga para una sola pantalla.

Estructura que debes copiar

HTML mínimo

```
<section class="stepper" id="projectStepper">
  <div class="stepper__indicators">
    <span class="is-active">1</span>
    <span>2</span>
    <span>3</span>
  </div>

  <article class="stepper__panel is-active" data-step="0">
    <h3>Paso 1: Datos básicos</h3>
    <input placeholder="Nombre del proyecto">
  </article>

  <article class="stepper__panel" data-step="1">
    <h3>Paso 2: Detalles</h3>
    <textarea placeholder="Descripción"></textarea>
  </article>

  <article class="stepper__panel" data-step="2">
    <h3>Paso 3: Confirmación</h3>
    <p>Revisa la información antes de enviar.</p>
  </article>

  <div class="stepper__actions">
    <button type="button" data-step-prev>Anterior</button>
    <button type="button" data-step-next>Siguiente</button>
  </div>
</section>
```

CSS necesario

```
.stepper__indicators {
  display: flex;
  gap: .5rem;
  margin-bottom: 1rem;
}

.stepper__indicators span {
  width: 32px;
  height: 32px;
  display: grid;
  place-items: center;
  border-radius: 999px;
  background: rgba(255, 255, 255, .12);
}

.stepper__indicators span.is-active {
  background: var(--primary, #2563eb);
  color: #fff;
}

.stepper__panel { display: none; }
.stepper__panel.is-active { display: block; }
.stepper__actions { display: flex; gap: .8rem; margin-top: 1rem; }
```

funciones.js - versión modular con export

```

export class Stepper {
  constructor(rootId) {
    this.root = document.getElementById(rootId);
    this.panels = this.root.querySelectorAll(".stepper__panel");
    this.indicators = this.root.querySelectorAll(".stepper__indicators span");
    this.prev = this.root.querySelector("[data-step-prev]");
    this.next = this.root.querySelector("[data-step-next]");
    this.current = 0;
    this.init();
  }

  init() {
    this.prev.addEventListener("click", () => this.goTo(this.current - 1));
    this.next.addEventListener("click", () => this.goTo(this.current + 1));
    this.render();
  }

  goTo(index) {
    if (index < 0 || index >= this.panels.length) return;
    this.current = index;
    this.render();
  }

  render() {
    this.panels.forEach((panel, index) => {
      panel.classList.toggle("is-active", index === this.current);
    });

    this.indicators.forEach((indicator, index) => {
      indicator.classList.toggle("is-active", index === this.current);
    });

    this.prev.disabled = this.current === 0;
    this.next.textContent = this.current === this.panels.length - 1 ? "Finalizar"
      : "Siguiente";
  }
}

```

script.js - importación e inicialización

```

import { Stepper } from "./funciones.js";

new Stepper("projectStepper");

```

script-normal.js - versión normal sin módulos

```

const stepper = document.getElementById("projectStepper");
const panels = stepper.querySelectorAll(".stepper__panel");
const indicators = stepper.querySelectorAll(".stepper__indicators span");
const prevButton = stepper.querySelector("[data-step-prev]");
const nextButton = stepper.querySelector("[data-step-next]");
let currentStep = 0;

function renderStepper() {
  panels.forEach((panel, index) => panel.classList.toggle("is-active", index ===
    currentStep));
  indicators.forEach((item, index) => item.classList.toggle("is-active", index ===
    currentStep));
}

prevButton.addEventListener("click", () => { currentStep = Math.max(0, currentStep -
  1); renderStepper(); });
nextButton.addEventListener("click", () => { currentStep = Math.min(panels.length - 1,
  currentStep + 1); renderStepper(); });
renderStepper();

```

Variables, clases e IDs que puedes cambiar

Elemento	Para qué sirve	Cómo adaptarlo
#projectStepper	Contenedor principal.	Cambia el id si tienes varios steppers.
.stepper__panel	Cada paso.	Duplica o elimina panels para más o menos pasos.

Elemento	Para qué sirve	Cómo adaptarlo
data-step-prev / data-step-next	Botones de navegación.	Deben estar dentro del stepper.

Paso a paso para reciclarlo en otro proyecto

1. Copia el contenedor .stepper completo.
2. Cambia títulos y campos dentro de cada .stepper__panel.
3. Agrega un indicador por cada panel.
4. Copia estilos de indicadores, panels y acciones.
5. Inicializa new Stepper("projectStepper").
6. Prueba avanzar, retroceder y llegar al último paso.

Qué debes revisar al integrarlo

- Que los ids del HTML coincidan con los ids usados en JavaScript.
- Que no estés usando el mismo id dos veces en la misma página.
- Que las clases activas como is-active, is-open, is-valid o is-invalid estén escritas igual en HTML, CSS y JS.
- Que si usas módulos el script se cargue con type="module".
- Que si usas la versión normal no dejes imports ni exports en el archivo.

Errores comunes

- Tener más indicadores que panels o al contrario.
- Poner los botones fuera del root y que querySelector no los encuentre.
- No validar cada paso antes de avanzar si el formulario es real.

Cómo cambiar nombres sin romper la funcionalidad

Paso	Qué hacer	Recomendación
1	Cambia primero el id en el HTML.	Ejemplo: loginForm por formAcceso.
2	Busca ese mismo id en script.js o script-normal.js.	Debe quedar exactamente igual.
3	Si el CSS usa una clase, cambia la clase en HTML y CSS al mismo tiempo.	No cambies solo uno de los dos archivos.
4	Si hay data-* como data-tab, data-step o data-value, revisa qué lo lee en JS.	Los atributos data-* suelen ser el puente entre HTML y JS.
5	Prueba la funcionalidad aislada antes de mezclarla con otros componentes.	Así sabes si el error viene del componente o de otra parte.

Adaptaciones comunes en proyectos reales

- Para una landing page: usa Stepper / Wizard dentro de una sección con clase .section y manten el diseño centrado.
- Para un dashboard: coloca Stepper / Wizard dentro de una card reutilizable y controla el estado con clases como is-active o is-open.
- Para un formulario: valida ids, required, minlength y data-error antes de enviar datos.
- Para un proyecto con muchos componentes: inicializa esta funcionalidad desde una función init21() o desde initApp().
- Para trabajo en equipo: deja comentarios cortos indicando qué HTML, CSS y JS pertenecen a Stepper / Wizard.

Mini guía de depuración si no funciona

1. Abre la consola del navegador con F12 y revisa si aparece un error rojo.
2. Si el error dice null, significa que JavaScript no encontró un elemento del DOM; revisa ids y clases.
3. Si el diseño no cambia, revisa que la clase que agrega JavaScript exista también en CSS.
4. Si usas import/export, abre el proyecto con Live Server; no lo abras solo con doble clic.
5. Si el componente depende de localStorage o sessionStorage, limpia el almacenamiento del navegador y prueba de nuevo.
6. Comenta temporalmente otros componentes para confirmar que no hay conflicto de nombres.

Patrón mental para reutilizarlo

Cada funcionalidad de esta guía se puede entender como una combinación de tres capas: la capa HTML define la estructura, la capa CSS define el diseño visual y la capa JavaScript controla el comportamiento. Para reciclarla bien, no copies solo una capa: copia la estructura mínima, los estilos necesarios y la inicialización correspondiente. Después, cambia nombres y valores de forma ordenada.

Consejo de adaptación profesional

En un wizard real, valida cada paso antes de permitir avanzar al siguiente.

22. Skeleton loader

¿Qué hace?

Muestra una estructura de carga temporal antes de renderizar datos reales. Es muy usado en páginas modernas para que la espera se sienta más profesional.

¿Cuándo usarlo?

Úsalo mientras esperas datos de una API, una búsqueda, cards de productos, listas o tablas.

Estructura que debes copiar

HTML mínimo

```
<section id="contentLoader" class="skeleton-grid">
  <article class="skeleton-card"></article>
  <article class="skeleton-card"></article>
  <article class="skeleton-card"></article>
</section>
```

CSS necesario

```
.skeleton-grid {
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(180px, 1fr));
  gap: 1rem;
}

.skeleton-card {
  height: 140px;
  border-radius: 1rem;
  background: linear-gradient(90deg, #e2e8f0 25%, #f8fafc 37%, #e2e8f0 63%);
  background-size: 400% 100%;
  animation: skeleton-loading 1.2s infinite;
}

@keyframes skeleton-loading {
  0% { background-position: 100% 0; }
  100% { background-position: 0 0; }
}
```

funciones.js - versión modular con export

```
export class SkeletonLoader {
  constructor(containerId) {
    this.container = document.getElementById(containerId);
  }

  show(count = 3) {
    this.container.innerHTML = Array.from({ length: count }, () => {
      return `<article class="skeleton-card"></article>`;
    }).join("");
  }

  hideWithContent(html) {
    this.container.innerHTML = html;
  }
}
```

script.js - importación e inicialización

```
import { SkeletonLoader } from "../funciones.js";

const loader = new SkeletonLoader("contentLoader");
loader.show(3);

setTimeout(() => {
  loader.hideWithContent(`
    <article class="card">Contenido cargado</article>
    <article class="card">Otro contenido</article>
  `);
}, 1500);
```

script-normal.js - versión normal sin módulos

```
const contentLoader = document.getElementById("contentLoader");

function showSkeleton(count = 3) {
  contentLoader.innerHTML = Array.from({ length: count }, () => {
    return `<article class="skeleton-card"></article>`;
  }).join("");
}

showSkeleton(3);
setTimeout(() => {
  contentLoader.innerHTML = `<article class="card">Contenido cargado</article>`;
}, 1500);
```

Variables, clases e IDs que puedes cambiar

Elemento	Para qué sirve	Cómo adaptarlo
count	Cantidad de placeholders.	Hazlo igual al número aproximado de cards reales.
.skeleton-card	Diseño del placeholder.	Cambia alto, radio o animación.
hideWithContent	Reemplaza skeleton por contenido real.	Pásale HTML seguro o creado con DOM.

Paso a paso para reciclarlo en otro proyecto

1. Agrega un contenedor donde irá la carga.
2. Copia el CSS del skeleton y keyframes.
3. Muestra skeleton antes de traer datos.
4. Cuando lleguen los datos, reemplaza con contenido real.
5. Prueba con setTimeout y luego cámbialo por fetch real.

Qué debes revisar al integrarlo

- Que los ids del HTML coincidan con los ids usados en JavaScript.
- Que no estés usando el mismo id dos veces en la misma página.
- Que las clases activas como is-active, is-open, is-valid o is-invalid estén escritas igual en HTML, CSS y JS.
- Que si usas módulos el script se cargue con type="module".
- Que si usas la versión normal no dejes imports ni exports en el archivo.

Errores comunes

- Dejar skeleton infinito si falla la API.
- Usar demasiados skeletons y saturar la pantalla.
- Renderizar HTML no confiable sin sanitizar.

Cómo cambiar nombres sin romper la funcionalidad

Paso	Qué hacer	Recomendación
1	Cambia primero el id en el HTML.	Ejemplo: loginForm por formAcceso.
2	Busca ese mismo id en script.js o script-normal.js.	Debe quedar exactamente igual.
3	Si el CSS usa una clase, cambia la clase en HTML y CSS al mismo tiempo.	No cambies solo uno de los dos archivos.
4	Si hay data-* como data-tab, data-step o data-value, revisa qué lo lee en JS.	Los atributos data-* suelen ser el puente entre HTML y JS.
5	Prueba la funcionalidad aislada antes de mezclarla con otros componentes.	Así sabes si el error viene del componente o de otra parte.

Adaptaciones comunes en proyectos reales

- Para una landing page: usa Skeleton loader dentro de una sección con clase .section y manten el diseño centrado.
- Para un dashboard: coloca Skeleton loader dentro de una card reutilizable y controla el estado con clases como is-active o is-open.
- Para un formulario: valida ids, required, minlength y data-error antes de enviar datos.
- Para un proyecto con muchos componentes: inicializa esta funcionalidad desde una función init22() o desde initApp().
- Para trabajo en equipo: deja comentarios cortos indicando qué HTML, CSS y JS pertenecen a Skeleton loader.

Mini guía de depuración si no funciona

1. Abre la consola del navegador con F12 y revisa si aparece un error rojo.
2. Si el error dice null, significa que JavaScript no encontró un elemento del DOM; revisa ids y clases.
3. Si el diseño no cambia, revisa que la clase que agrega JavaScript exista también en CSS.
4. Si usas import/export, abre el proyecto con Live Server; no lo abras solo con doble clic.
5. Si el componente depende de localStorage o sessionStorage, limpia el almacenamiento del navegador y prueba de nuevo.
6. Comenta temporalmente otros componentes para confirmar que no hay conflicto de nombres.

Patrón mental para reutilizarlo

Cada funcionalidad de esta guía se puede entender como una combinación de tres capas: la capa HTML define la estructura, la capa CSS define el diseño visual y la capa JavaScript controla el comportamiento. Para reciclarla bien, no copies solo una capa: copia la estructura mínima, los estilos necesarios y la inicialización correspondiente. Después, cambia nombres y valores de forma ordenada.

Consejo de adaptación profesional

Incluye estado de error si la API falla; no dejes el skeleton cargando para siempre.

23. Select personalizado

¿Qué hace?

Reemplaza el select nativo por un selector bonito, controlable y editable. Permite opciones con estilo, flecha personalizada, blur y mejor integración visual.

¿Cuándo usarlo?

Úsalo cuando el select nativo se vea mal o no combine con el diseño, como en tarjetas glassmorphism.

Estructura que debes copiar

HTML mínimo

```
<div class="custom-select" id="statusSelect">
  <input type="hidden" id="statusValue" value="pendiente">

  <button type="button" class="custom-select__button">
    <span class="custom-select__value">Pendiente</span>
    <span class="custom-select__arrow">▼</span>
  </button>

  <ul class="custom-select__options">
    <li class="custom-select__option is-selected"
      data-value="pendiente">Pendiente</li>
    <li class="custom-select__option" data-value="proceso">En proceso</li>
    <li class="custom-select__option" data-value="finalizado">Finalizado</li>
  </ul>
</div>
```

CSS necesario

```
.custom-select {
  position: relative;
  width: 100%;
  z-index: 20;
}

.custom-select__button {
  width: 100%;
  display: flex;
  justify-content: space-between;
  align-items: center;
  padding: .9rem 1rem;
  border-radius: 1rem;
  border: 1px solid rgba(255, 255, 255, .25);
  background: rgba(255, 255, 255, .14);
  color: inherit;
  cursor: pointer;
}

.custom-select__options {
  position: absolute;
  top: calc(100% + .5rem);
  left: 0;
  width: 100%;
  padding: .5rem;
  margin: 0;
  list-style: none;
  border-radius: 1rem;
  background: rgba(15, 23, 42, .88);
  backdrop-filter: blur(18px);
  opacity: 0;
  transform: translateY(-8px);
  pointer-events: none;
  transition: .2s ease;
}

.custom-select.is-open .custom-select__options {
  opacity: 1;
  transform: translateY(0);
  pointer-events: auto;
}

.custom-select__option {
  padding: .75rem .9rem;
  border-radius: .8rem;
  cursor: pointer;
}

.custom-select__option:hover,
.custom-select__option.is-selected {
  background: linear-gradient(135deg, #2563eb, #7c3aed);
}
```

funciones.js - versión modular con export

```

export class CustomSelect {
  constructor(rootId) {
    this.root = document.getElementById(rootId);
    this.hiddenInput = this.root.querySelector("input[type='hidden']");
    this.button = this.root.querySelector(".custom-select__button");
    this.valueText = this.root.querySelector(".custom-select__value");
    this.options = this.root.querySelectorAll(".custom-select__option");
    this.init();
  }

  init() {
    this.button.addEventListener("click", () =>
      this.root.classList.toggle("is-open"));

    this.options.forEach((option) => {
      option.addEventListener("click", () => this.select(option));
    });

    document.addEventListener("click", (event) => {
      if (!this.root.contains(event.target))
        this.root.classList.remove("is-open");
    });
  }

  select(option) {
    this.hiddenInput.value = option.dataset.value;
    this.valueText.textContent = option.textContent;
    this.options.forEach((item) => item.classList.remove("is-selected"));
    option.classList.add("is-selected");
    this.root.classList.remove("is-open");
    this.hiddenInput.dispatchEvent(new Event("change"));
  }
}

```

script.js - importación e inicialización

```

import { CustomSelect } from "../funciones.js";

new CustomSelect("statusSelect");

// Para leer el valor:
const status = document.getElementById("statusValue").value;

```

script-normal.js - versión normal sin módulos

```

const customSelect = document.getElementById("statusSelect");
const customButton = customSelect.querySelector(".custom-select__button");
const customValue = customSelect.querySelector(".custom-select__value");
const hiddenInput = customSelect.querySelector("input[type='hidden']");
const customOptions = customSelect.querySelectorAll(".custom-select__option");

customButton.addEventListener("click", () =>
  customSelect.classList.toggle("is-open"));
customOptions.forEach((option) => {
  option.addEventListener("click", () => {
    hiddenInput.value = option.dataset.value;
    customValue.textContent = option.textContent;
    customSelect.classList.remove("is-open");
  });
});

```

Variables, clases e IDs que puedes cambiar

Elemento	Para qué sirve	Cómo adaptarlo
#statusSelect	Contenedor del selector.	Cámbialo por unidad, categoría, ciudad, etc.
#statusValue	Input oculto con el valor real.	Tu JS debe leer este value.
data-value	Valor lógico de cada opción.	Cámbialo según lo que tu app necesite.
.custom-select__value	Texto visible actual.	Debe coincidir con la opción inicial.

Paso a paso para reciclarlo en otro proyecto

1. Copia el bloque HTML completo del custom select.
2. Cambia el id del contenedor y del input hidden.
3. Agrega, elimina o edita `li.custom-select__option`.
4. Cada `li` debe tener `data-value` y texto visible.
5. Copia el CSS del selector.
6. Inicializa `new CustomSelect("idDelSelector")`.
7. Lee el valor desde el input hidden con `.value`.

Qué debes revisar al integrarlo

- Que los ids del HTML coincidan con los ids usados en JavaScript.
- Que no estés usando el mismo id dos veces en la misma página.
- Que las clases activas como `is-active`, `is-open`, `is-valid` o `is-invalid` estén escritas igual en HTML, CSS y JS.
- Que si usas módulos el script se cargue con `type="module"`.
- Que si usas la versión normal no dejes `imports` ni `exports` en el archivo.

Errores comunes

- Leer el texto del botón en vez del input oculto.
- Olvidar `data-value` en las opciones.
- No subir `z-index` y que el menú quede detrás de otra tarjeta.

Cómo cambiar nombres sin romper la funcionalidad

Paso	Qué hacer	Recomendación
1	Cambia primero el id en el HTML.	Ejemplo: <code>loginForm</code> por <code>formAcceso</code> .
2	Busca ese mismo id en <code>script.js</code> o <code>script-normal.js</code> .	Debe quedar exactamente igual.
3	Si el CSS usa una clase, cambia la clase en HTML y CSS al mismo tiempo.	No cambies solo uno de los dos archivos.
4	Si hay <code>data-*</code> como <code>data-tab</code> , <code>data-step</code> o <code>data-value</code> , revisa qué lo lee en JS.	Los atributos <code>data-*</code> suelen ser el puente entre HTML y JS.
5	Prueba la funcionalidad aislada antes de mezclarla con otros componentes.	Así sabes si el error viene del componente o de otra parte.

Adaptaciones comunes en proyectos reales

- Para una landing page: usa Select personalizado dentro de una sección con clase `.section` y manten el diseño centrado.
- Para un dashboard: coloca Select personalizado dentro de una card reutilizable y controla el estado con clases como `is-active` o `is-open`.
- Para un formulario: valida ids, `required`, `minlength` y `data-error` antes de enviar datos.
- Para un proyecto con muchos componentes: inicializa esta funcionalidad desde una función `init23()` o desde `initApp()`.
- Para trabajo en equipo: deja comentarios cortos indicando qué HTML, CSS y JS pertenecen a Select personalizado.

Mini guía de depuración si no funciona

1. Abre la consola del navegador con F12 y revisa si aparece un error rojo.
2. Si el error dice `null`, significa que JavaScript no encontró un elemento del DOM; revisa ids y clases.
3. Si el diseño no cambia, revisa que la clase que agrega JavaScript exista también en CSS.

4. Si usas import/export, abre el proyecto con Live Server; no lo abras solo con doble clic.
5. Si el componente depende de localStorage o sessionStorage, limpia el almacenamiento del navegador y prueba de nuevo.
6. Comenta temporalmente otros componentes para confirmar que no hay conflicto de nombres.

Patrón mental para reutilizarlo

Cada funcionalidad de esta guía se puede entender como una combinación de tres capas: la capa HTML define la estructura, la capa CSS define el diseño visual y la capa JavaScript controla el comportamiento. Para reciclarla bien, no copies solo una capa: copia la estructura mínima, los estilos necesarios y la inicialización correspondiente. Después, cambia nombres y valores de forma ordenada.

Consejo de adaptación profesional

Mantén un input hidden porque facilita leer el valor desde formularios y desde JavaScript.

24. Tabs / pestañas

¿Qué hace?

Permite cambiar entre paneles internos sin recargar la página. Se usa en perfiles, productos, dashboards, documentación y configuraciones.

¿Cuándo usarlo?

Úsalo cuando varias secciones están relacionadas pero no deben verse todas al mismo tiempo.

Estructura que debes copiar

HTML mínimo

```
<div class="tabs" id="productTabs">
  <div class="tabs__buttons">
    <button class="is-active" data-tab="description">Descripción</button>
    <button data-tab="details">Detalles</button>
    <button data-tab="reviews">Reseñas</button>
  </div>

  <section class="tab-panel is-active" id="description">Contenido de
    descripción</section>
  <section class="tab-panel" id="details">Contenido de detalles</section>
  <section class="tab-panel" id="reviews">Contenido de reseñas</section>
</div>
```

CSS necesario

```
.tabs__buttons {
  display: flex;
  flex-wrap: wrap;
  gap: .5rem;
  margin-bottom: 1rem;
}

.tabs__buttons button {
  width: auto;
  padding: .75rem 1rem;
  border-radius: 999px;
  background: rgba(255, 255, 255, .12);
  color: inherit;
}

.tabs__buttons button.is-active {
  background: linear-gradient(135deg, #2563eb, #7c3aed);
  color: #fff;
}

.tab-panel {
  display: none;
  padding: 1rem;
  border-radius: 1rem;
  background: rgba(255, 255, 255, .08);
}

.tab-panel.is-active { display: block; }
```

funciones.js - versión modular con export

```
export class Tabs {
  constructor(rootId) {
    this.root = document.getElementById(rootId);
    this.buttons = this.root.querySelectorAll("[data-tab]");
    this.panels = this.root.querySelectorAll(".tab-panel");
    this.init();
  }

  init() {
    this.buttons.forEach((button) => {
      button.addEventListener("click", () => this.activate(button.dataset.tab));
    });
  }

  activate(tabId) {
    this.buttons.forEach((button) => {
      button.classList.toggle("is-active", button.dataset.tab === tabId);
    });

    this.panels.forEach((panel) => {
      panel.classList.toggle("is-active", panel.id === tabId);
    });
  }
}
```

script.js - importación e inicialización

```
import { Tabs } from "../funciones.js";

new Tabs("productTabs");
```

script-normal.js - versión normal sin módulos

```
const tabs = document.getElementById("productTabs");
const tabButtons = tabs.querySelectorAll("[data-tab]");
const tabPanels = tabs.querySelectorAll(".tab-panel");

tabButtons.forEach((button) => {
  button.addEventListener("click", () => {
    const tabId = button.dataset.tab;
    tabButtons.forEach((btn) => btn.classList.toggle("is-active", btn ===
      button));
    tabPanels.forEach((panel) => panel.classList.toggle("is-active", panel.id ===
      tabId));
  });
});
```

Variables, clases e IDs que puedes cambiar

Elemento	Para qué sirve	Cómo adaptarlo
data-tab	Nombre del panel que se activará.	Debe coincidir con el id del panel.
.is-active	Clase que muestra botón y panel activos.	No cambies el nombre si no ajustas JS/CSS.
#productTabs	Contenedor raíz.	Permite tener varias zonas de tabs independientes.

Paso a paso para reciclarlo en otro proyecto

1. Copia la estructura .tabs completa.
2. Cambia textos de botones y contenido de paneles.
3. Asegúrate de que data-tab="x" coincida con id="x".
4. Copia los estilos de botones y paneles.
5. Inicializa new Tabs("productTabs").
6. Prueba todos los botones y verifica que solo haya un panel activo.

Qué debes revisar al integrarlo

- Que los ids del HTML coincidan con los ids usados en JavaScript.
- Que no estés usando el mismo id dos veces en la misma página.
- Que las clases activas como is-active, is-open, is-valid o is-invalid estén escritas igual en HTML, CSS y JS.
- Que si usas módulos el script se cargue con type="module".
- Que si usas la versión normal no dejes imports ni exports en el archivo.

Errores comunes

- data-tab no coincide con ningún id.
- No poner is-active inicial y que no se vea nada.
- Usar ids repetidos en varias secciones de tabs.

Cómo cambiar nombres sin romper la funcionalidad

Paso	Qué hacer	Recomendación
1	Cambia primero el id en el HTML.	Ejemplo: loginForm por formAcceso.
2	Busca ese mismo id en script.js o script-normal.js.	Debe quedar exactamente igual.
3	Si el CSS usa una clase, cambia la clase en HTML y CSS al mismo tiempo.	No cambies solo uno de los dos archivos.
4	Si hay data-* como data-tab, data-step o data-value, revisa qué lo lee en JS.	Los atributos data-* suelen ser el puente entre HTML y JS.
5	Prueba la funcionalidad aislada antes de mezclarla con otros componentes.	Así sabes si el error viene del componente o de otra parte.

Adaptaciones comunes en proyectos reales

- Para una landing page: usa Tabs / pestañas dentro de una sección con clase .section y manten el diseño centrado.
- Para un dashboard: coloca Tabs / pestañas dentro de una card reutilizable y controla el estado con clases como is-active o is-open.
- Para un formulario: valida ids, required, minlength y data-error antes de enviar datos.
- Para un proyecto con muchos componentes: inicializa esta funcionalidad desde una función init24() o desde initApp().
- Para trabajo en equipo: deja comentarios cortos indicando qué HTML, CSS y JS pertenecen a Tabs / pestañas.

Mini guía de depuración si no funciona

1. Abre la consola del navegador con F12 y revisa si aparece un error rojo.
2. Si el error dice null, significa que JavaScript no encontró un elemento del DOM; revisa ids y clases.
3. Si el diseño no cambia, revisa que la clase que agrega JavaScript exista también en CSS.
4. Si usas import/export, abre el proyecto con Live Server; no lo abras solo con doble clic.
5. Si el componente depende de localStorage o sessionStorage, limpia el almacenamiento del navegador y prueba de nuevo.
6. Comenta temporalmente otros componentes para confirmar que no hay conflicto de nombres.

Patrón mental para reutilizarlo

Cada funcionalidad de esta guía se puede entender como una combinación de tres capas: la capa HTML define la estructura, la capa CSS define el diseño visual y la capa JavaScript controla el comportamiento. Para reciclarla bien, no copies solo una capa: copia la estructura mínima, los estilos necesarios y la inicialización correspondiente. Después, cambia nombres y valores de forma ordenada.

Consejo de adaptación profesional

Si las tabs cambian contenido importante, considera actualizar la URL con hash para compartir el estado.

25. Acordeón / FAQ

¿Qué hace?

Abre y cierra bloques de contenido. Es perfecto para preguntas frecuentes, secciones de ayuda, detalles de producto y menús informativos.

¿Cuándo usarlo?

Úsalo cuando tienes mucho contenido textual y quieres ahorrar espacio visual.

Estructura que debes copiar

HTML mínimo

```
<section class="accordion" id="faqAccordion">
  <article class="accordion__item">
    <button type="button" class="accordion__button">¿Qué incluye el
      servicio?</button>
    <div class="accordion__content">
      <p>Incluye diseño, desarrollo y soporte inicial.</p>
    </div>
  </article>

  <article class="accordion__item">
    <button type="button" class="accordion__button">¿Cuánto tarda?</button>
    <div class="accordion__content">
      <p>Depende del alcance del proyecto.</p>
    </div>
  </article>
</section>
```

CSS necesario

```
.accordion__item {
  border-bottom: 1px solid rgba(255, 255, 255, .15);
}

.accordion__button {
  width: 100%;
  padding: 1rem;
  border: 0;
  background: transparent;
  color: inherit;
  text-align: left;
  font-weight: 700;
  cursor: pointer;
}

.accordion__content {
  max-height: 0;
  overflow: hidden;
  transition: max-height .25s ease;
}

.accordion__item.is-open .accordion__content {
  max-height: 220px;
}

.accordion__content p {
  padding: 0 1rem 1rem;
  margin: 0;
  color: var(--text-muted, #cbd5e1);
}
```

funciones.js - versión modular con export

```
export class Accordion {
  constructor(rootId, allowMultiple = false) {
    this.root = document.getElementById(rootId);
    this.items = this.root.querySelectorAll(".accordion__item");
    this.allowMultiple = allowMultiple;
    this.init();
  }

  init() {
    this.items.forEach((item) => {
      const button = item.querySelector(".accordion__button");
      button.addEventListener("click", () => this.toggle(item));
    });
  }

  toggle(selectedItem) {
    if (!this.allowMultiple) {
      this.items.forEach((item) => {
        if (item !== selectedItem) item.classList.remove("is-open");
      });
    }

    selectedItem.classList.toggle("is-open");
  }
}
```

script.js - importación e inicialización

```
import { Accordion } from "./funciones.js";

new Accordion("faqAccordion", false);
```

script-normal.js - versión normal sin módulos

```
const faqAccordion = document.getElementById("faqAccordion");
const accordionItems = faqAccordion.querySelectorAll(".accordion__item");

accordionItems.forEach((item) => {
  const button = item.querySelector(".accordion__button");
  button.addEventListener("click", () => {
    accordionItems.forEach((otherItem) => {
      if (otherItem !== item) otherItem.classList.remove("is-open");
    });
    item.classList.toggle("is-open");
  });
});
```

Variables, clases e IDs que puedes cambiar

Elemento	Para qué sirve	Cómo adaptarlo
#faqAccordion	Contenedor del acordeón.	Cambia el id si tienes varios acordeones.
.accordion__item	Cada bloque desplegable.	Duplica este article para más preguntas.
allowMultiple	Permite abrir varios bloques a la vez.	true abre varios; false solo uno.
max-height	Altura máxima del contenido abierto.	Auméntala si el contenido es largo.

Paso a paso para reciclarlo en otro proyecto

1. Copia el section.accordion completo.
2. Duplica article.accordion__item para cada pregunta o bloque.
3. Cambia el texto del botón y del contenido.
4. Copia el CSS y ajusta max-height si hace falta.
5. Inicializa new Accordion("faqAccordion", false).
6. Prueba abrir una pregunta y confirmar que las demás se cierran.

Qué debes revisar al integrarlo

- Que los ids del HTML coincidan con los ids usados en JavaScript.
- Que no estés usando el mismo id dos veces en la misma página.
- Que las clases activas como is-active, is-open, is-valid o is-invalid estén escritas igual en HTML, CSS y JS.
- Que si usas módulos el script se cargue con type="module".
- Que si usas la versión normal no dejes imports ni exports en el archivo.

Errores comunes

- Poner contenido muy largo y que se corte por max-height.
- No usar button y perder accesibilidad.
- Olvidar que allowMultiple cambia el comportamiento.

Cómo cambiar nombres sin romper la funcionalidad

Paso	Qué hacer	Recomendación
1	Cambia primero el id en el HTML.	Ejemplo: loginForm por formAcceso.
2	Busca ese mismo id en script.js o script-normal.js.	Debe quedar exactamente igual.
3	Si el CSS usa una clase, cambia la clase en HTML y CSS al mismo tiempo.	No cambies solo uno de los dos archivos.
4	Si hay data-* como data-tab, data-step o data-value, revisa qué lo lee en JS.	Los atributos data-* suelen ser el puente entre HTML y JS.
5	Prueba la funcionalidad aislada antes de mezclarla con otros componentes.	Así sabes si el error viene del componente o de otra parte.

Adaptaciones comunes en proyectos reales

- Para una landing page: usa Acordeón / FAQ dentro de una sección con clase .section y manten el diseño centrado.
- Para un dashboard: coloca Acordeón / FAQ dentro de una card reutilizable y controla el estado con clases como is-active o is-open.
- Para un formulario: valida ids, required, minlength y data-error antes de enviar datos.
- Para un proyecto con muchos componentes: inicializa esta funcionalidad desde una función init25() o desde initApp().
- Para trabajo en equipo: deja comentarios cortos indicando qué HTML, CSS y JS pertenecen a Acordeón / FAQ.

Mini guía de depuración si no funciona

1. Abre la consola del navegador con F12 y revisa si aparece un error rojo.
2. Si el error dice null, significa que JavaScript no encontró un elemento del DOM; revisa ids y clases.
3. Si el diseño no cambia, revisa que la clase que agrega JavaScript exista también en CSS.
4. Si usas import/export, abre el proyecto con Live Server; no lo abras solo con doble clic.
5. Si el componente depende de localStorage o sessionStorage, limpia el almacenamiento del navegador y prueba de nuevo.
6. Comenta temporalmente otros componentes para confirmar que no hay conflicto de nombres.

Patrón mental para reutilizarlo

Cada funcionalidad de esta guía se puede entender como una combinación de tres capas: la capa HTML define la estructura, la capa CSS define el diseño visual y la capa JavaScript controla el comportamiento. Para reciclarla bien, no copies solo una capa: copia la estructura mínima, los estilos necesarios y la inicialización correspondiente. Después, cambia nombres y valores de forma ordenada.

Consejo de adaptación profesional

Para accesibilidad avanzada, puedes agregar aria-expanded al botón y actualizarlo con JavaScript.

Apéndice: mapa rápido de integración

#	Funcionalidad	Archivos	Versión JS	Revisión clave
10	Contador animado	HTML + CSS + JS	Modular o normal	Revisar ids, clases y data-*
11	Login demo	HTML + CSS + JS	Modular o normal	Revisar ids, clases y data-*
12	Logout demo	HTML + CSS + JS	Modular o normal	Revisar ids, clases y data-*
13	Recordar usuario	HTML + CSS + JS	Modular o normal	Revisar ids, clases y data-*
14	Password toggle	HTML + CSS + JS	Modular o normal	Revisar ids, clases y data-*
15	Validación de formulario	HTML + CSS + JS	Modular o normal	Revisar ids, clases y data-*
16	Mensajes por campo	HTML + CSS + JS	Modular o normal	Revisar ids, clases y data-*
17	Character counter	HTML + CSS + JS	Modular o normal	Revisar ids, clases y data-*
18	Range preview	HTML + CSS + JS	Modular o normal	Revisar ids, clases y data-*
19	File info	HTML + CSS + JS	Modular o normal	Revisar ids, clases y data-*
20	Autosave draft	HTML + CSS + JS	Modular o normal	Revisar ids, clases y data-*
21	Stepper / Wizard	HTML + CSS + JS	Modular o normal	Revisar ids, clases y data-*
22	Skeleton loader	HTML + CSS + JS	Modular o normal	Revisar ids, clases y data-*
23	Select personalizado	HTML + CSS + JS	Modular o normal	Revisar ids, clases y data-*
24	Tabs / pestañas	HTML + CSS + JS	Modular o normal	Revisar ids, clases y data-*
25	Acordeón / FAQ	HTML + CSS + JS	Modular o normal	Revisar ids, clases y data-*

Checklist final antes de copiar a otro proyecto

- Copiar solo la funcionalidad necesaria para no llenar el proyecto de código que no vas a usar.
- Mantener una sola estrategia de JavaScript: modular o normal.
- Si usas módulos, index.html debe tener script type="module".
- Si usas script normal, elimina export, import y clases que no inicialices.
- Revisar en móvil, tablet y escritorio.
- Probar estados vacíos, errores, valores válidos y valores extremos.
- Cambiar nombres de localStorage para evitar choques entre proyectos.
- Usar nombres claros para ids y clases: loginForm, productTabs, faqAccordion, etc.
- Comentar el código si lo vas a entregar como tarea o compartir con el equipo.